

트리 (Tree)

강 지 훈

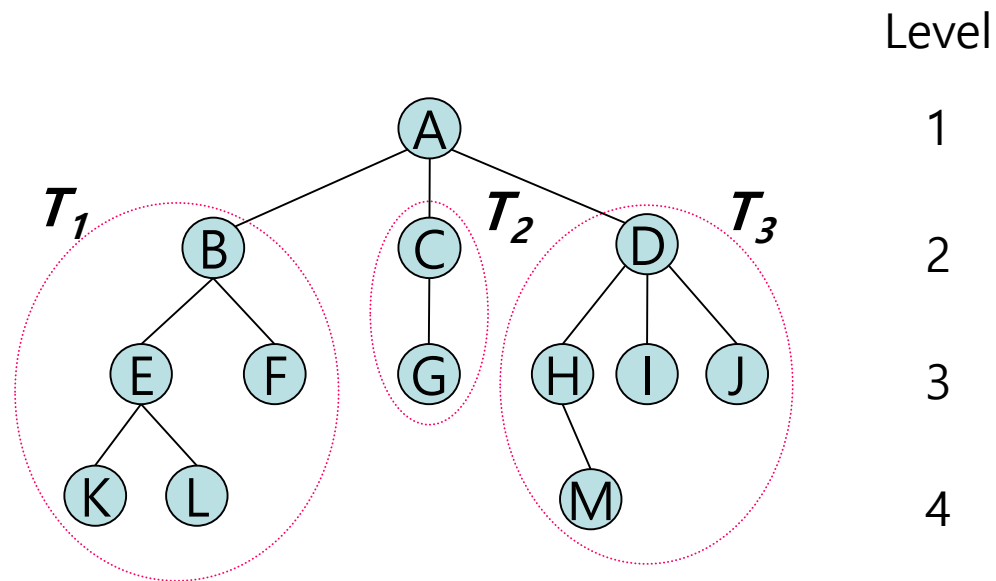
jhkang@cnu.ac.kr



트리 (Tree)

□ 트리

- **트리(tree)** 는 하나 이상의 노드(node)로 구성되는 유한집합으로, 다음의 조건을 만족해야 한다:
 1. **루트(root)** 라 부르는 특별히 지정된 노드가 하나 있다.
 2. 나머지 노드들은 n 개 ($n \geq 0$)의 노드가 서로 중복되지 않는 집합 (disjoint sets) $T_1, \dots, T_n$ 으로 나누어지며, 이 각각의 노드 집합은 **트리** 이어야 한다. $T_1, \dots, T_n$ 이들을 루트의 **부트리(subtrees)** 라 한다.
- 재귀적으로 정의: 부트리는 다시 트리로 정의되고 있다.



□ 용어 [1]

- **노드(node)**: 정보 항목 (**item**) 과 다른 노드로의 가지 (**branches**)로 구성되는 트리의 구성 단위
- **노드의 디그리 (degree of a node)**: 그 노드의 서브트리의 개수
 - 예: $\text{degree}(A) = 3, \text{degree}(M) = 0$
- **트리의 디그리 (degree of a tree)**: 트리에 있는 노드들의 디그리 중에서 가장 큰 값
- **잎(leaf) (또는 끝(terminal))** : 디그리가 0인 노드
 - Example: K, L, F, G, M, I, J
- **노드 X의 자식 (child of a node X)**: X의 서브트리의 루트
- **부모(parent)** : 자식의 역관계
 - 예: B 는 E 와 F의 부모.

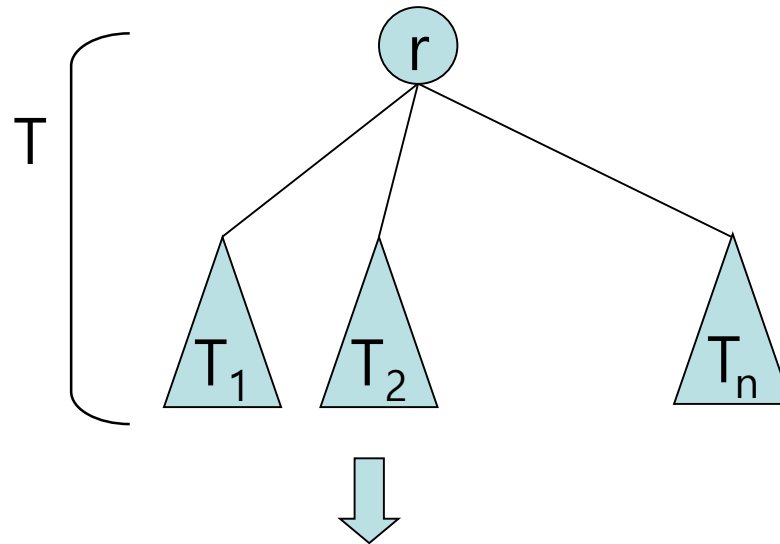
□ 용어 [2]

- 형제 (sibling): 부모가 동일한 노드
 - Example: H, I, J are siblings
- 노드 X 의 조상 (ancestor): 노드 X 에서부터 루트까지의 경로를 따라 존재하는 노드
 - 예: M의 조상은 A, D, H 이다.
- 레벨 (level):
 - 루트의 레벨은 1 이다.
 - 루트의 자식의 레벨은 2 이다.
 - 노드 X의 레벨이 k 이면, X의 자식의 레벨은 (k+1) 이다.
- 높이 (height) (또는 깊이 (depth)): 트리의 최대 레벨
 - 예:
 - ◆ M의 레벨 = 4 : (트리에서 최대값)
 - ◆ 트리의 높이 = 4

트리의 표현

□ 리스트를 이용한 표현

- 부트리(subtree) 들을 리스트로 표현

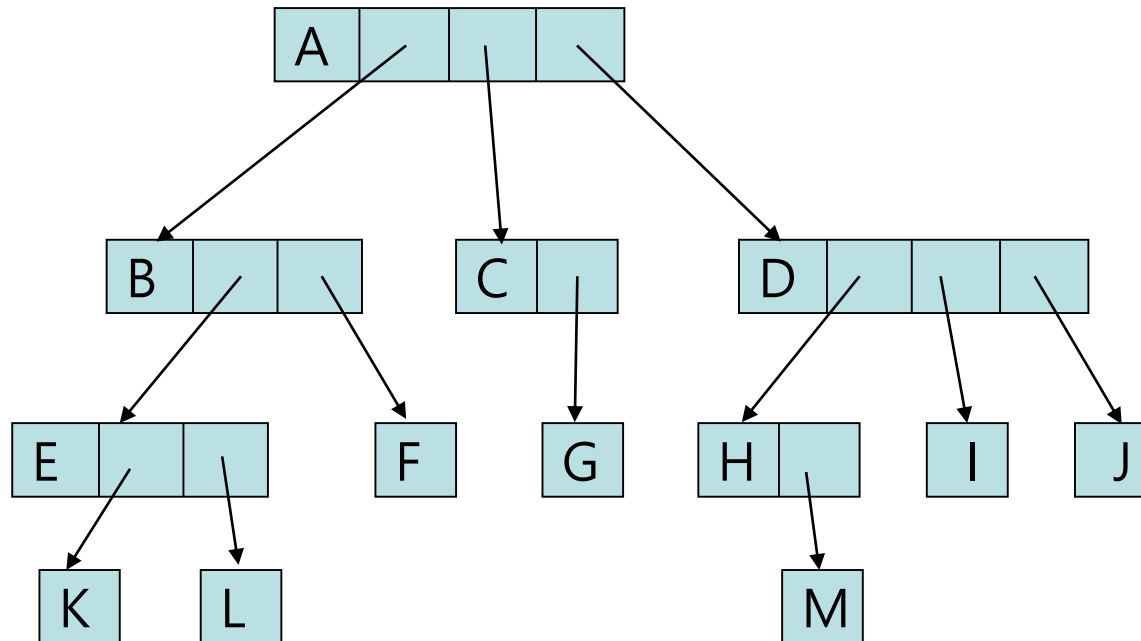


$$(T) = (r (T_1, T_2, \dots, T_n))$$

□ 리스트를 이용한 표현

■ 예: 가변길이 노드 (variable length nodes)를 사용하여 리스트를 표현

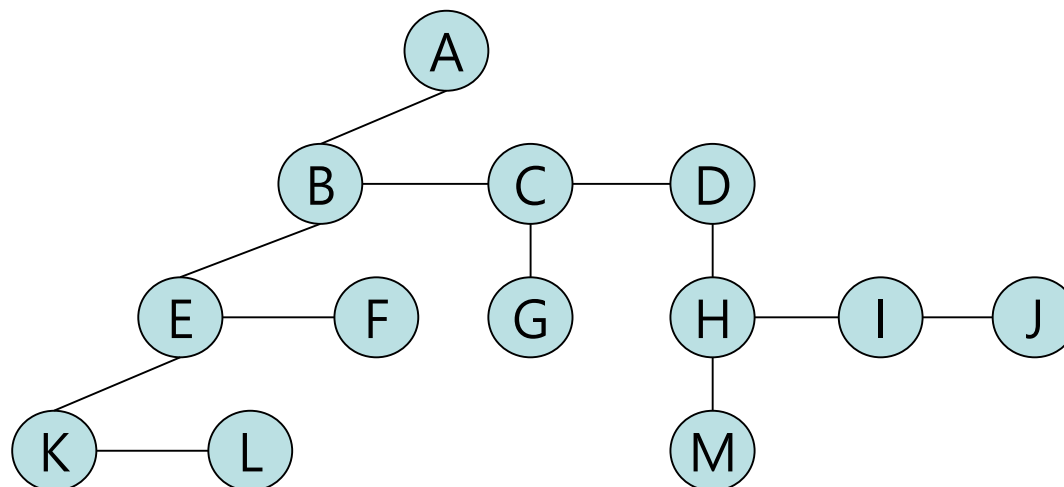
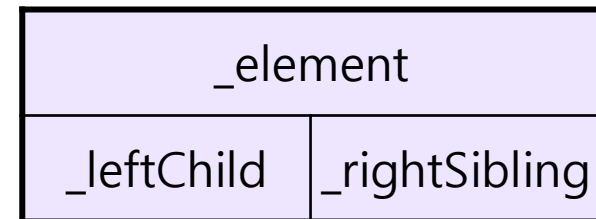
● (A (B (E (K, L), F), C (G), D (H (M), I, J)))



□ 왼쪽 자식-오른쪽 형제 (Left Child - Right Sibling) 표현

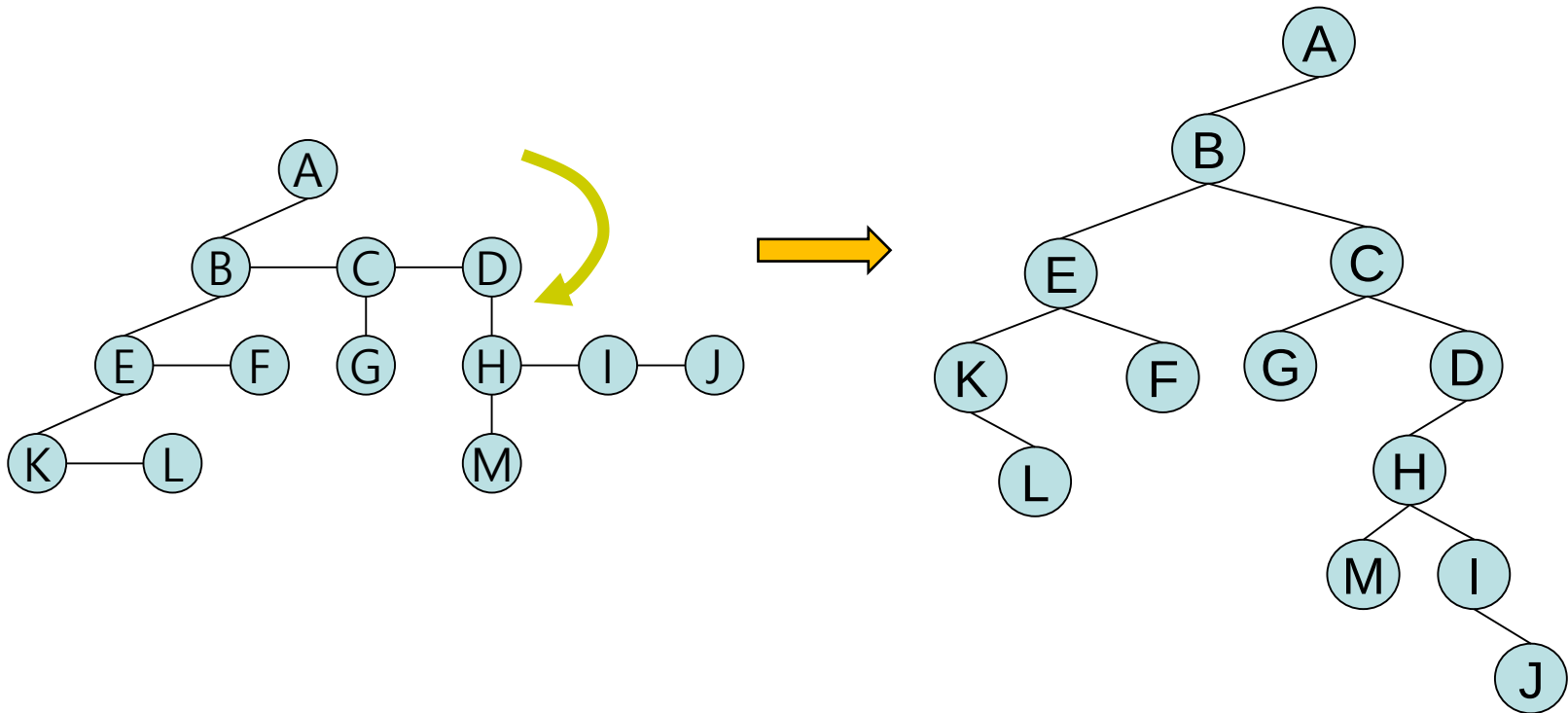
- 고정길이 노드 사용
- 각 노드는 3 개의 속성이 필요

```
public class TreeNode<T> {
    private Element      _element ;
    private TreeNode<T>  _leftChild ;
    private TreeNode<T>  _rightSibling ;
    .....
}
```



□ 이진 트리로 표현

- Left child - right sibling 트리를 시계 방향으로 45 도 회전시킨다.
- 결국 이진트리(binary tree) 가 된다.



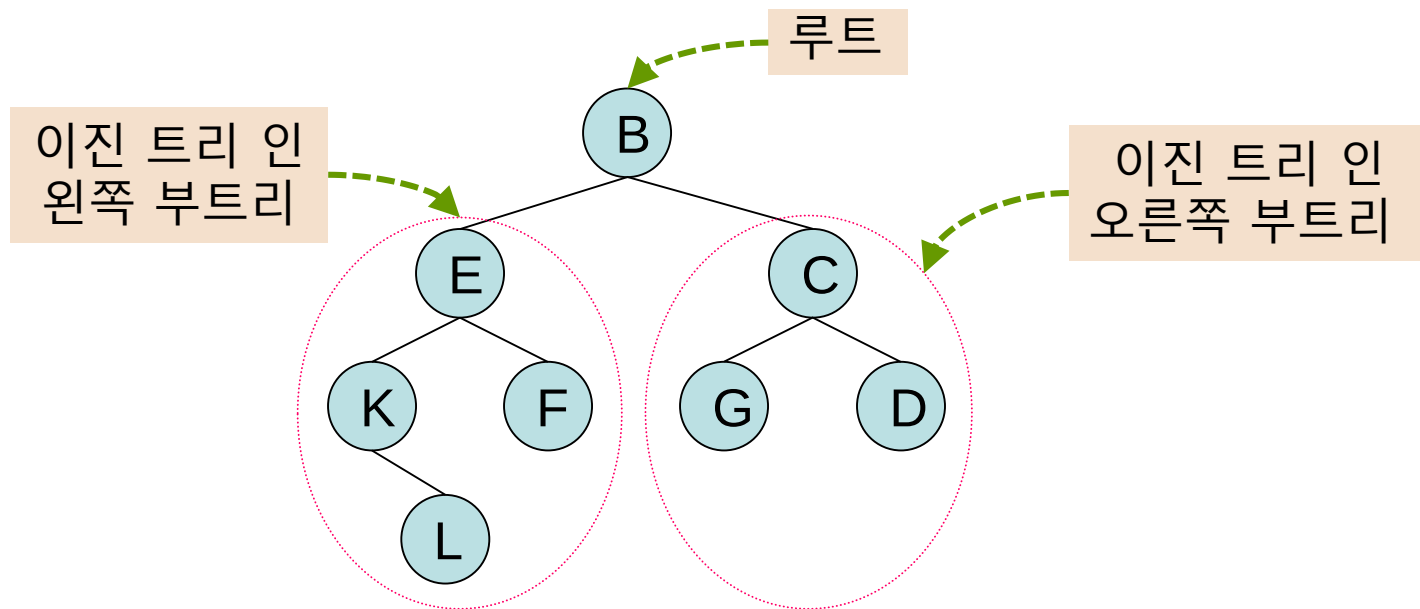
이진 트리 (Binary Tree)



□ 이진트리 (Binary Trees)

■ 이진트리 (binary tree) 는 노드의 유한집합으로 다음의 조건을 만족해야 한다:

- 1) 비어 있거나, 또는
- 2) 루트(root) 와 두개의 서로 겹치지 않는 이진 트리로 구성되며, 각각 왼쪽부트리 (left subtree), 오른쪽부트리 (right subtree)라 한다.

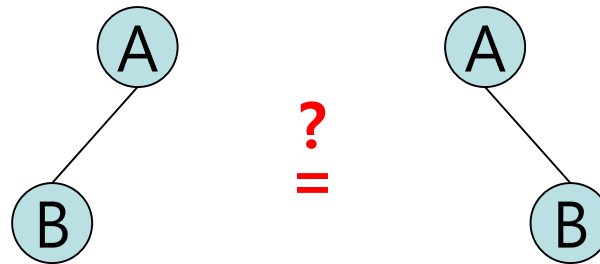


■ 이진 트리에서도 트리의 용어를 그대로 사용

□ 트리와 이진트리의 차이점?

■ 서브트리의 순서

- 이진 트리에는 서브트리의 순서 (위치) 가 있다.
그러나 트리에는 순서가 없다.



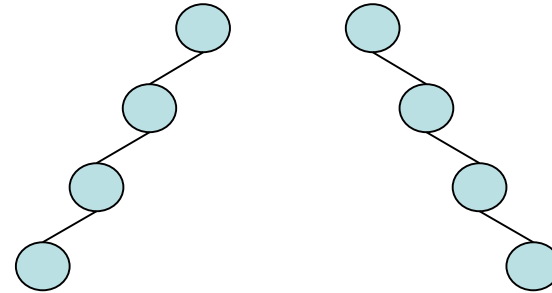
- 이들이 트리 라면, 이 둘은 같다.
- 이들이 이진트리 라면, 이 둘은 다르다.

■ 노드의 최소 개수 (이론적 관점)

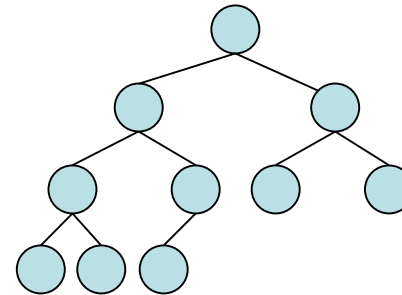
- 트리에는 적어도 하나의 노드가 존재한다.
- 이진트리에는 노드가 하나도 없을 수도 있다.

□ 특수한 이진 트리

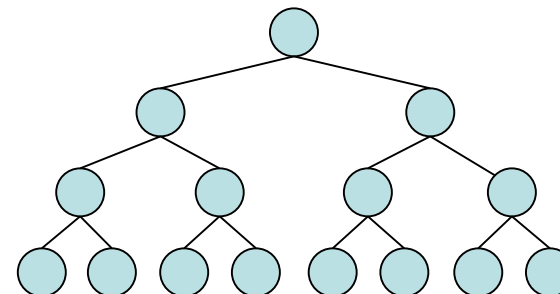
- 치우친 이진트리
(Skewed binary tree
/ Degenerate binary tree)



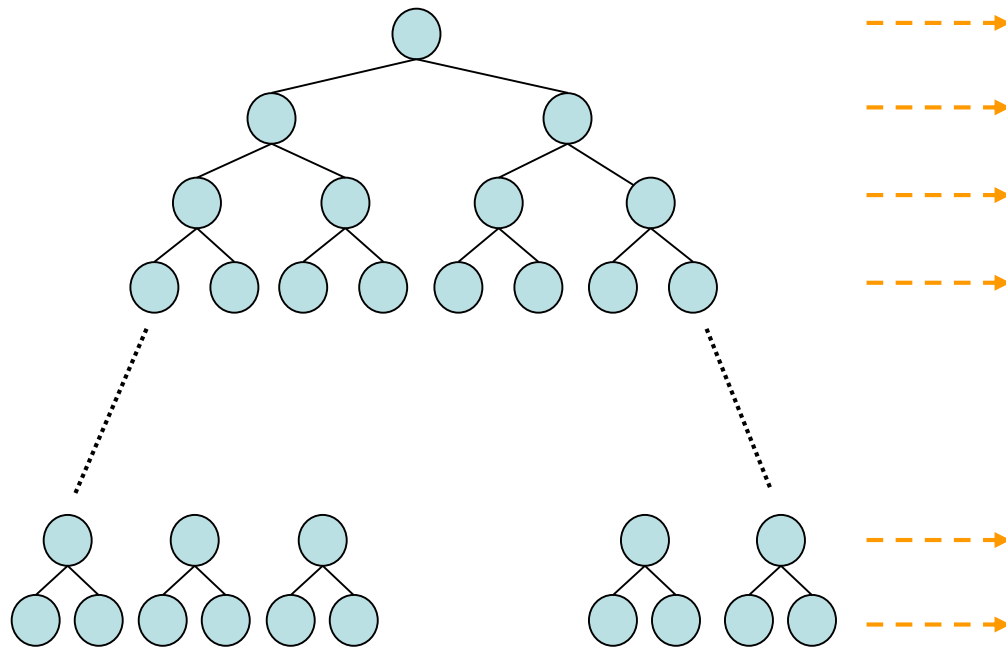
- 완전 이진트리
(Complete binary tree)



- 꼭찬 이진트리
(Full binary tree)



□ 레벨과 노드 수와의 관계



Level i	2^{i-1}	$2^i - 1$
1	2^0	$2^1 - 1$
2	2^1	$2^2 - 1$
3	2^2	$2^3 - 1$
4	2^3	$2^4 - 1$
$k-1$	2^{k-2}	$2^{k-1} - 1$
k	2^{k-1}	$2^k - 1$

■ 레벨 i 에 있을 수 있는 노드의 최대 개수
 $\Rightarrow 2^{i-1} (i \geq 1)$

■ 깊이가 k 인 이진트리가 가질 수 있는 노드의 최대 개수
 $\Rightarrow 2^0 + 2^1 + \dots + 2^{k-1} = 2^k - 1 (k \geq 1)$

□ 잎 노드 수와 디그리 2 인 노드 수와의 관계

■ Relationship between number of leaf nodes and nodes of degree 2.

- Let n_0 : # of leaf nodes, and
 n_2 : # of nodes of degree 2.
 Then, $n_0 = n_2 + 1$.

(Proof)

Let n : # of nodes in the tree, and
 n_1 : # of nodes of degree 1.

Then $n = n_0 + n_1 + n_2$[1]

Let B : # of branches in the tree.

Then

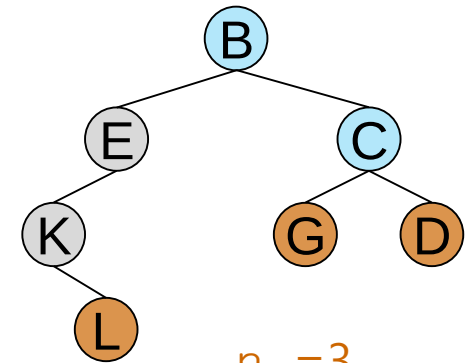
$$n = B + 1.$$

$$B = 1 \cdot n_1 + 2 \cdot n_2.$$

$$\text{So, } n = 1 \cdot n_1 + 2 \cdot n_2 + 1 \text{[2]}$$

$$\text{Since [1]=[2], } n = n_0 + n_1 + n_2 = 1 \cdot n_1 + 2 \cdot n_2 + 1.$$

Therefore, $n_0 = n_2 + 1$. ■



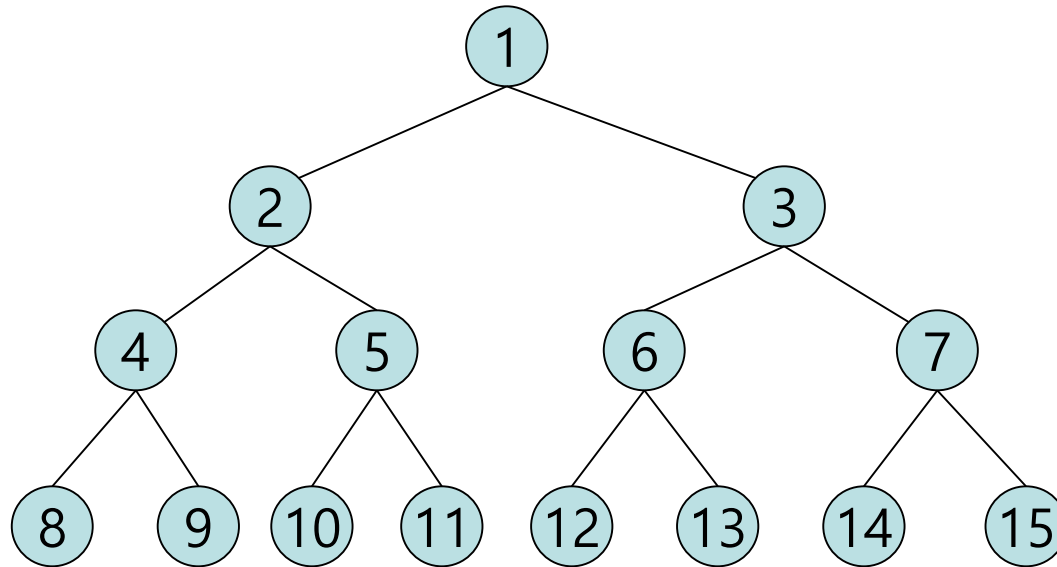
$$\begin{aligned} n_0 &= 3 \\ n_1 &= 2 \\ n_2 &= 2 \end{aligned}$$

□ 꼭찬 이진트리 (Full Binary Trees)

■ A **full binary tree** of Depth k :

A binary tree of depth k having $2^k - 1$ nodes, ($k \geq 0$)

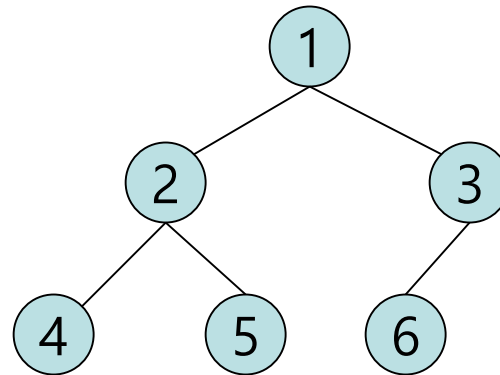
● Example: Full binary tree of depth 4.



□ 완전 이진트리 (Complete binary trees)

- 노드가 n 개인 완전 이진트리:
 - (n 개 이상의 노드를 가지고 있는) 꽉찬 이진트리에서 노드 번호가 1번부터 n 번까지의 노드를 가지고 있는 트리

- 예: 노드가 6 개인 완전 이진트리



- 꽉찬 이진트리는 완전 이진트리이다.

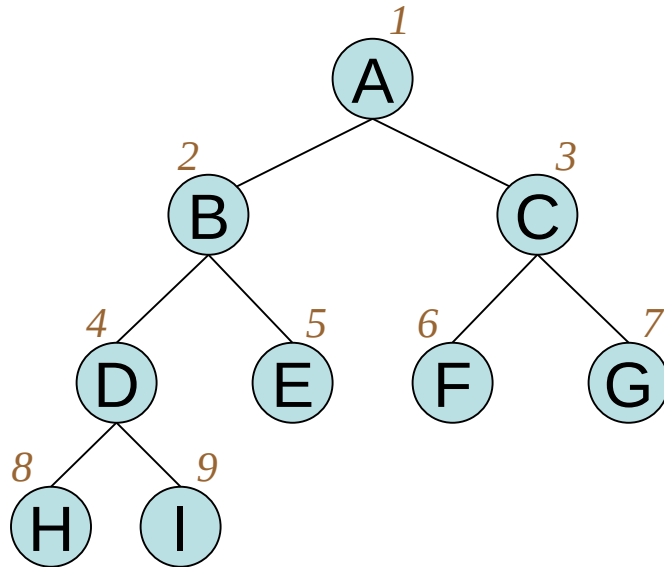
이진트리의 표현

배열을 이용
연결 체인을 이용

배열을 이용한 표현

□ 배열을 이용한 표현 [1]

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	A	B	C	D	E	F	G	H	I



$\text{parent}(6) \rightarrow \lfloor 6/2 \rfloor = 3$
 $\text{parent}(9) \rightarrow \lfloor 9/2 \rfloor = 4$
 $\text{parent}(1) \rightarrow \text{none}$

$\text{lchild}(2) \rightarrow 2 \cdot 2 = 4$

$\text{lchild}(3) \rightarrow 2 \cdot 3 = 6$

$\text{lchild}(7) \rightarrow \text{none}$

$\text{rchild}(2) \rightarrow 2 \cdot 2 + 1 = 5$

$\text{rchild}(3) \rightarrow 2 \cdot 3 + 1 = 7$

$\text{rchild}(7) \rightarrow = \text{none}$

□ 노드 번호와 배열 인덱스의 관계

- 노드의 개수가 n 인 완전 이진트리를 배열을 이용하여 표현하면 다음의 관계가 성립한다:

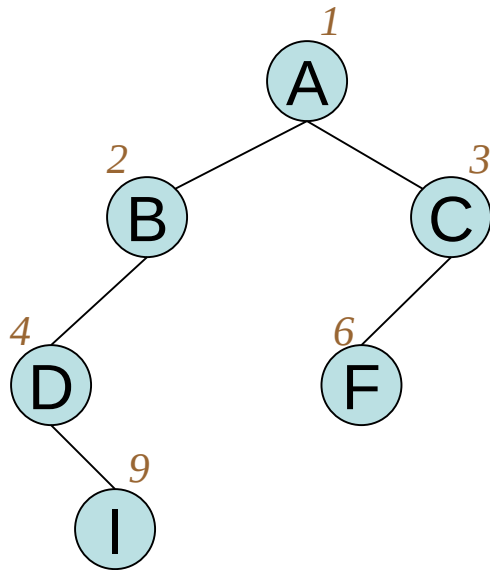
배열 인덱스가 i ($1 \leq i \leq n$) 인 노드 x 에 대해,

1. $i \neq 1$ 이면, x 의 부모는 $\lfloor i/2 \rfloor$ 에 존재
 $i = 1$ 이면, x 는 루트 노드이므로, 부모가 존재하지 않음
2. $2 \cdot i \leq n$ 이라면, x 의 왼쪽 자식은 $2 \cdot i$ 에 존재
 $2 \cdot i > n$ 이라면, x 는 왼쪽 자식이 없음
3. $2 \cdot i + 1 \leq n$ 이라면, x 의 오른쪽 자식은 $2 \cdot i + 1$ 에 존재
 $2 \cdot i + 1 > n$ 이라면, x 는 오른쪽 자식이 없음

- 노드의 개수가 n 인 완전 이진트리의 깊이: $\lfloor \log_2 n \rfloor + 1$

□ 일반 이진트리를 배열로 표현 [1]

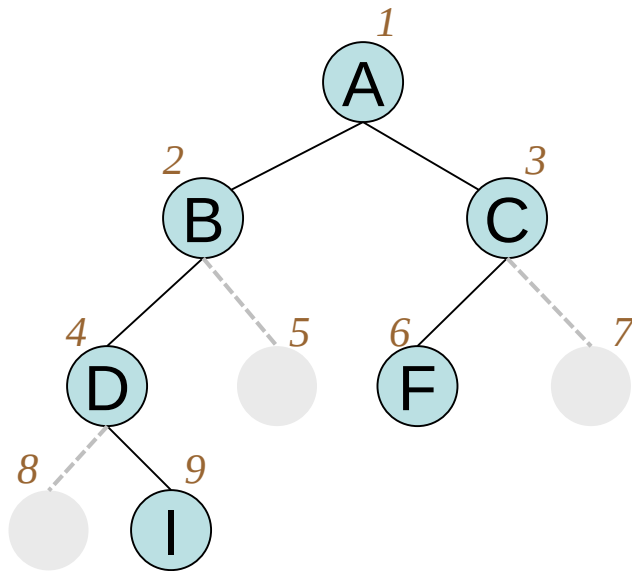
■ 일반 이진트리에서의 노드 번호



[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	A	B	C	D		F			I

□ 일반 이진트리를 배열로 표현 [2]

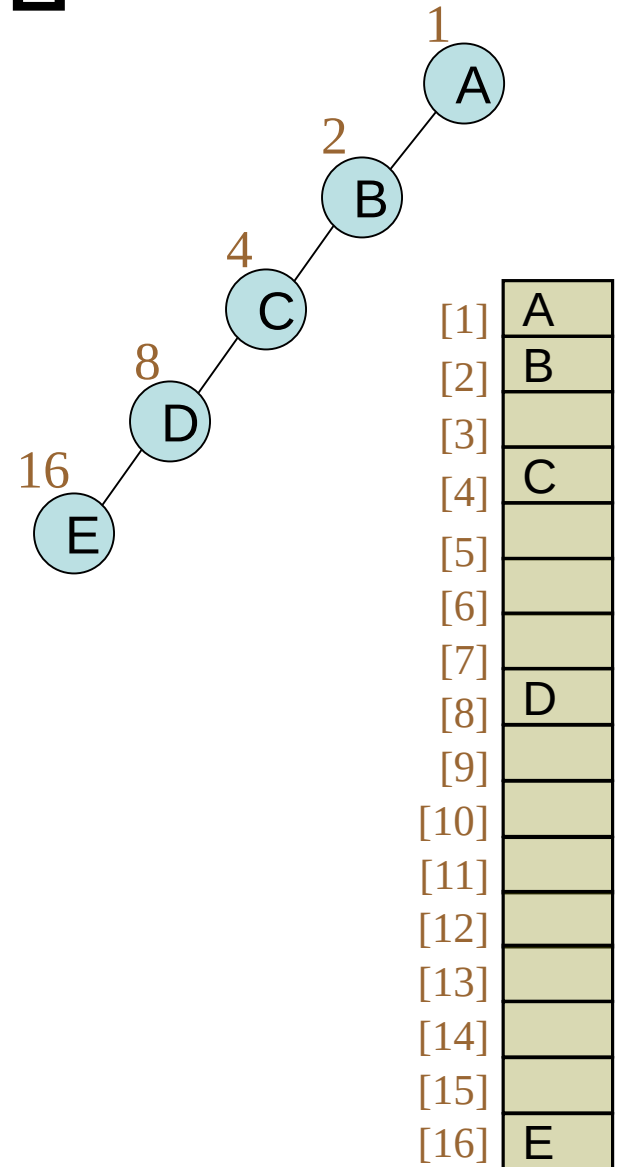
■ 일반 이진트리에서의 노드 번호



[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	A	B	C	D		F			I

□ 배열을 이용할 경우의 장단점

- 어떠한 이진트리도 배열을 이용하여 표현 가능
 - 메모리 낭비가 많을 수 있다.
- 완전 이진트리에서는:
 - 메모리 낭비 공간이 없음
- 트리에서 삽입과 삭제가 자주 발생할 경우
 - 위치를 바꾸어야 할 노드의 수가 많을 수 있다 ➡ 비효율적
 - 연결 체인을 이용한 표현을 사용하는 것이 더 좋음

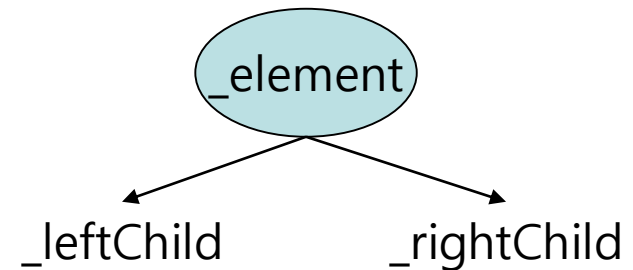
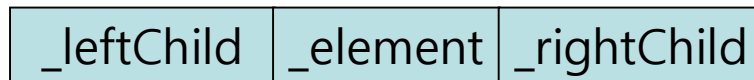


연결 체인을 이용한 표현

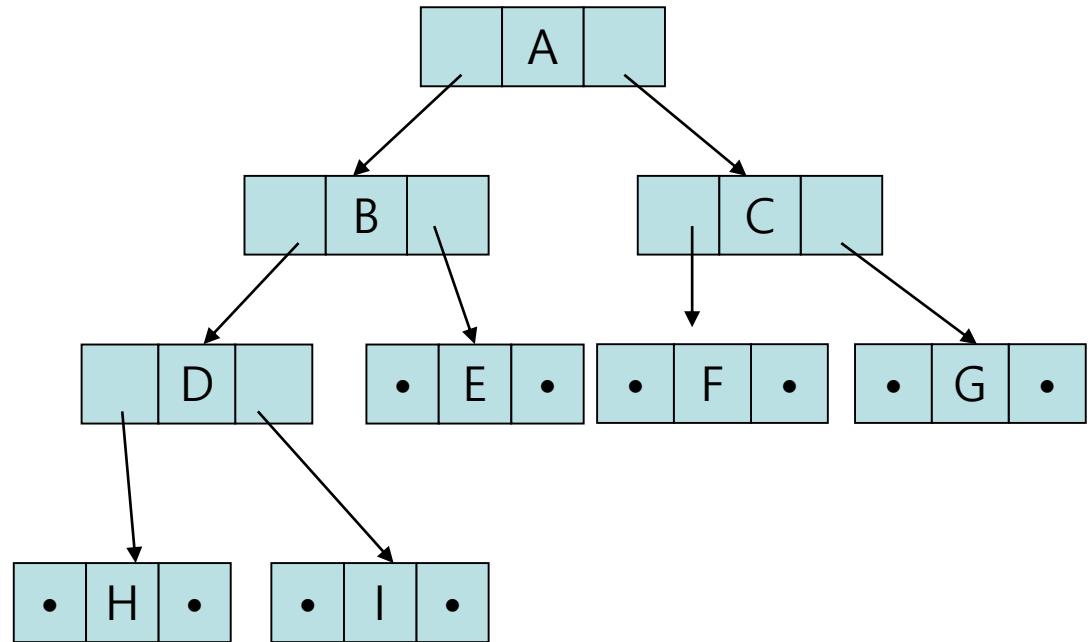
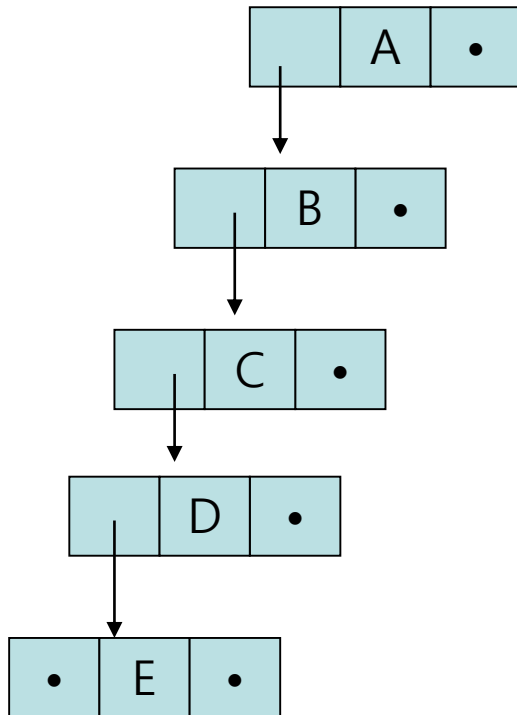
Class "BinaryNode"

□ 연결체인을 사용한 노드의 구현 구조

```
public class BinaryNode<T>
{
    // 비공개 멤버 변수
    private T _element ;
    private BinaryNode<T> _leftChild ;
    private BinaryNode<T> _rightChild ;
}
```



예제



□ Class "BinaryNode<T>"의 공개함수

■ BinaryNode 객체 사용법

- public BinaryNode () ;
- public BinaryNode (T givenElement,
BinaryNode<T> givenLeftChild,
BinaryNode<T> givenRightChild) ;
- public boolean hasLeftChild() ;
- public boolean hasRightChild() ;
- public boolean isLeaf() ;
- public T element() ;
- public void setElement (T newElement) ;
- public BinaryNode<T> leftChild() ;
- public void setLeftChild (BinaryNode<T> newLeftChild) ;
- public BinaryNode<T> rightChild() ;
- public void setRightChild (BinaryNode<T> newRightChild) ;

□ Class “BinaryNode”의 공개함수

■ BinaryNode 객체 사용법을 Java로 구체적으로 표현

- public BinaryNode () ;
- public BinaryNode (T anElement,
BinaryNode<T> aLeftChild,
BinaryNode<T> aRightChild) ;
- public boolean hasLeftChild() ;
- public boolean hasRightChild() ;
- public boolean isLeaf() ;
- public T element() ;
- public void setElement (T newElement) ;
- public BinaryNode<T> leftChild() ;
- public void setLeftChild (BinaryNode<T> newLeftChild) ;
- public BinaryNode<T> rightChild() ;
- public void setRightChild (BinaryNode<T> newRightChild) ;

❑ Class "BinaryNode<T>"의 구현: 인스턴스 변수

```
public class BinaryNode<T>
{
    // 비공개 멤버 변수
    private T _element ;
    private BinaryNode<T> _leftChild ;
    private BinaryNode<T> _rightChild ;
}
```


□ Class "BinaryNode<T>"의 구현: 생성자

```

public class BinaryNode<T>
{
    // 비공개 멤버 변수
    .....

    // 생성자
    public BinaryNode ( )
    {
        this._element = null ;
        this._leftChild = null ;
        this._rightChild = null ;
    }

    public BinaryNode ( T                givenElement,
                        BinaryNode<T>    givenLeftChild,
                        BinaryNode<T>    givenRightChild)
    {
        this._element = givenElement ;
        this._leftChild = givenLeftChild ;
        this._rightChild = givenRightChild ;
    }
}

```

□ BinaryNode<T> : 상태 알아보기

```
public class BinaryNode<T>
{
    // 비공개 멤버 변수
    .....

    // 왼쪽 자식을 가지고 있는지 확인
    public boolean hasLeftChild()
    {
        return (this._leftChild != null) ;
    }

    // 오른쪽 자식을 가지고 있는지 확인
    public boolean hasRightChild()
    {
        return (this._rightChild != null) ;
    }

    // Leaf인지 확인
    public boolean isLeaf()
    {
        return (this._leftChild == null) && (this._rightChild == null) ;
    }
}
```

□ BinaryNode<T>: Getter/Setter [1]

```
public class BinaryNode<T>
{
    // 비공개 멤버 변수
    .....

    // 원소 얻어내기: getter for element
    public T element()
    {
        return this._element ;
    }

    // 원소 설정하기: setter for element
    public void setElement (T newElement)
    {
        this._element = newElement ;
    }
}
```

□ BinaryNode<T>: Getter/Setter [2]

```
public class BinaryNode<T>
{
    // 비공개 멤버 변수
    .....

    // left의 자식 Node를 얻어냄: getter for leftChild
    public BinaryNode<T> leftChild()
    {
        return this._leftChild ;
    }

    // left의 자식을 설정함: setter for leftChild
    public void setLeftChild(BinaryNode<T> newLeftChild)
    {
        this._leftChild = newLeftChild ;
    }
}
```

□ BinaryNode : Getter/Setter [3]

```
public class BinaryNode<T>
{
    // 비공개 멤버 변수
    .....

    // right의 자식 Node를 얻어냄: getter for rightChild
    public BinaryNode<T> rightChild()
    {
        return this._rightChild ;
    }

    // right의 자식을 설정함: setter for rightChild
    public void setRightChild (BinaryNode<T> newRightChild)
    {
        this._rightChild = newRightChild ;
    }
}
```

이진 트리 탐색

(Binary Tree Traversals)



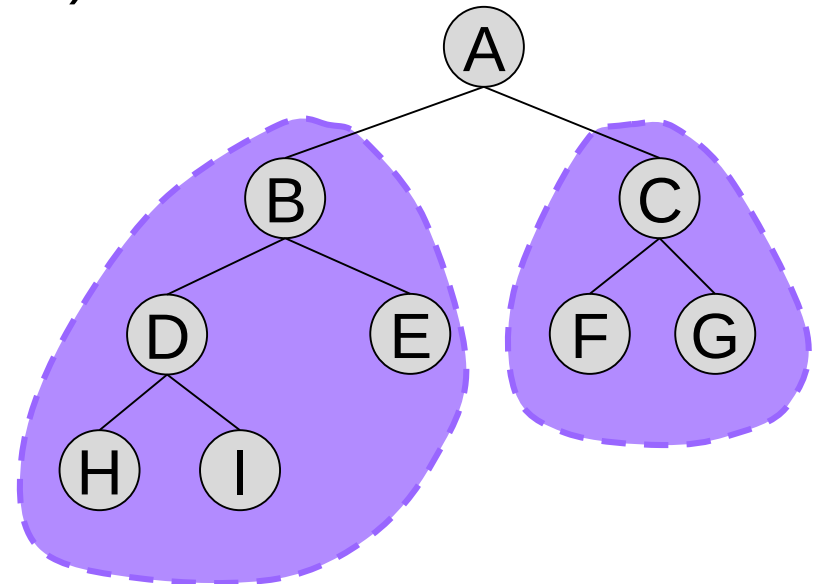
이진트리 탐색

체계적으로 모든 노드를 방문

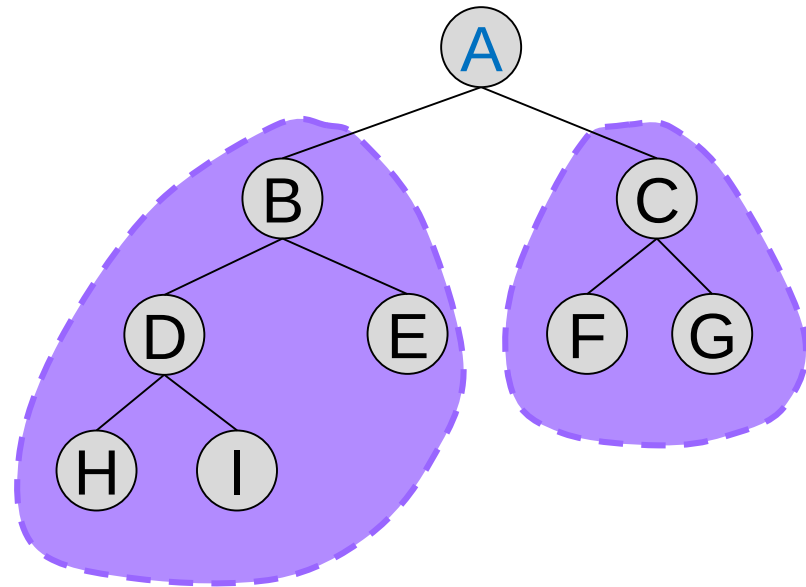
- 중위 탐색 (Inorder traversal)
- 전위 탐색 (Preorder traversal)
- 후위 탐색 (Postorder traversal)

중위탐색 (Inorder Traversal) 이란?

- 왼쪽 부트리를
중위 탐색하여
모든 노드를 방문
- 루트를 방문
- 오른쪽 부트리를
중위 탐색하여
모든 노드를 방문

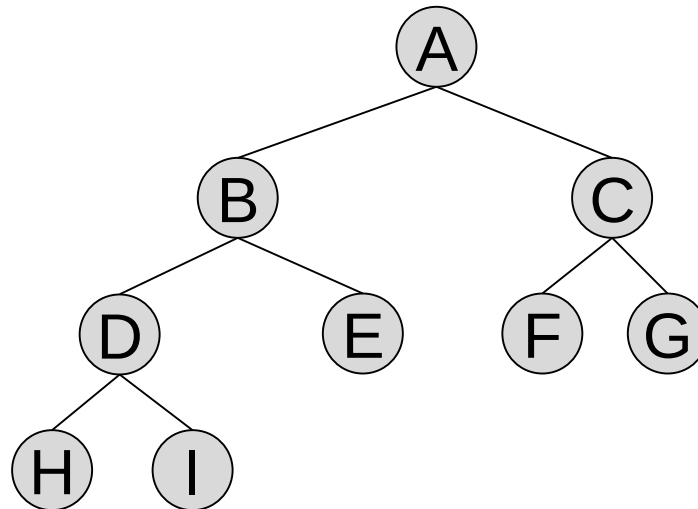


□ 이진트리 탐색 순서에서 루트의 위치는?

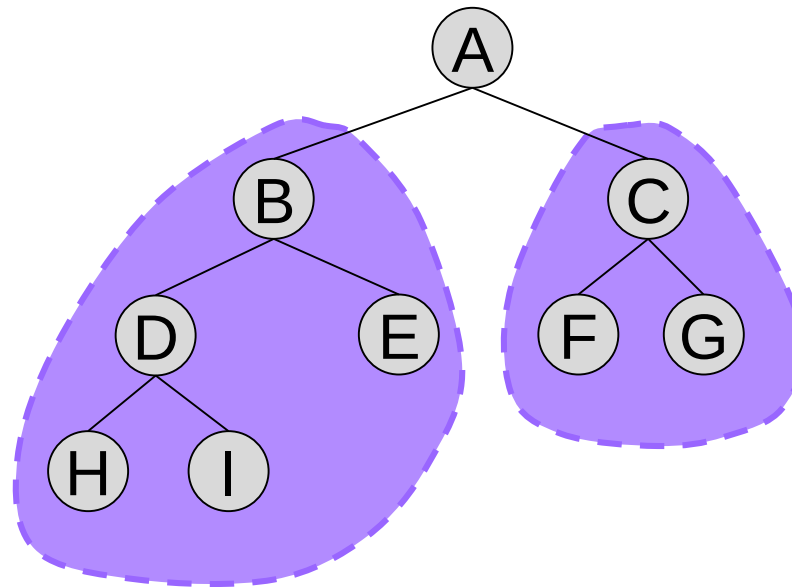


- Inorder : (((H)–D–(I))–B–(E))–A–((F)–C–(G))
H – D – I – B – E – A – F – C – G
- Preorder : A – B – D – H – I – E – C – F – G
- Postorder : H – I – D – E – B – F – G – C – A

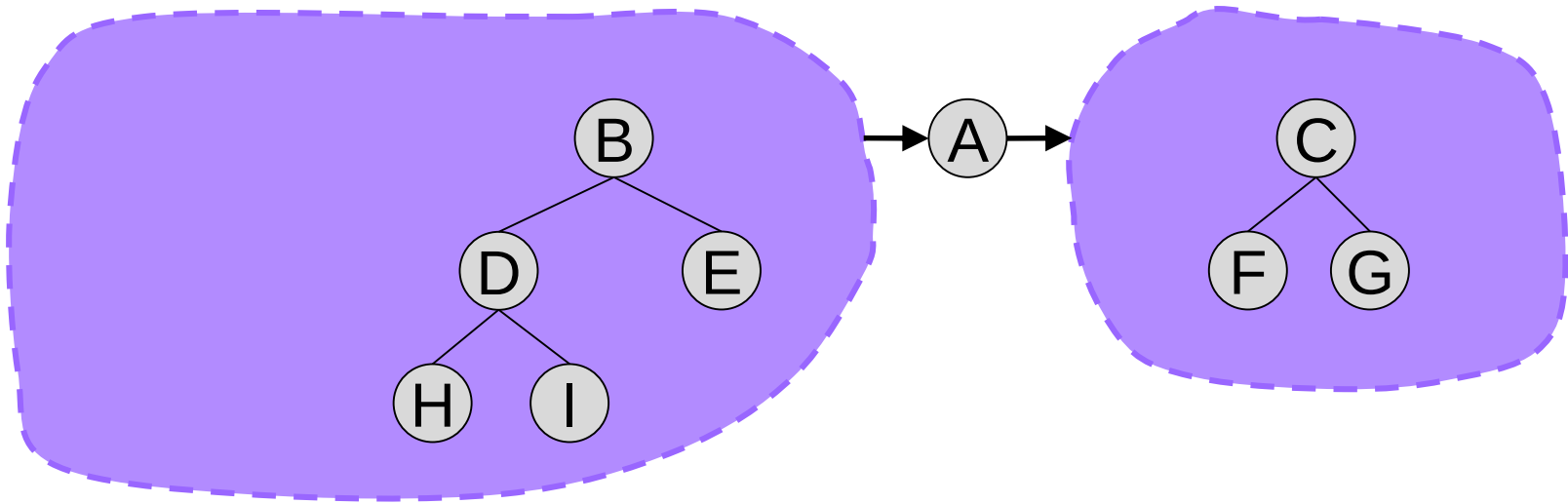
□ 예제: 중위 탐색 (Inorder) [0]



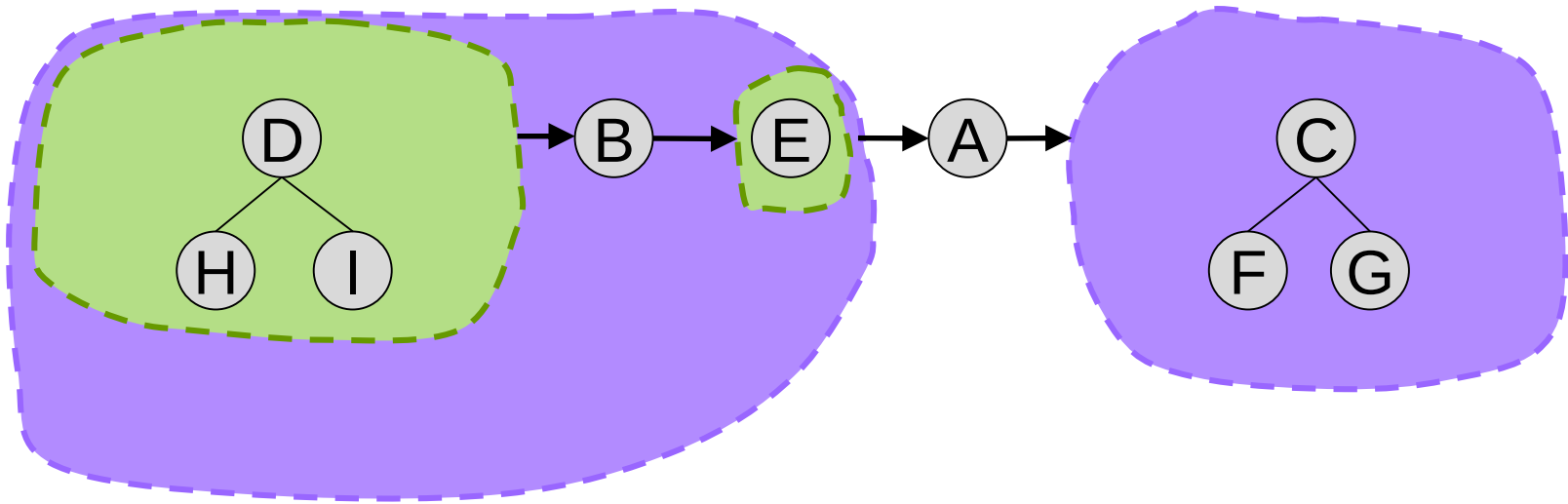
□ 예제: 중위 탐색 (Inorder) [1]



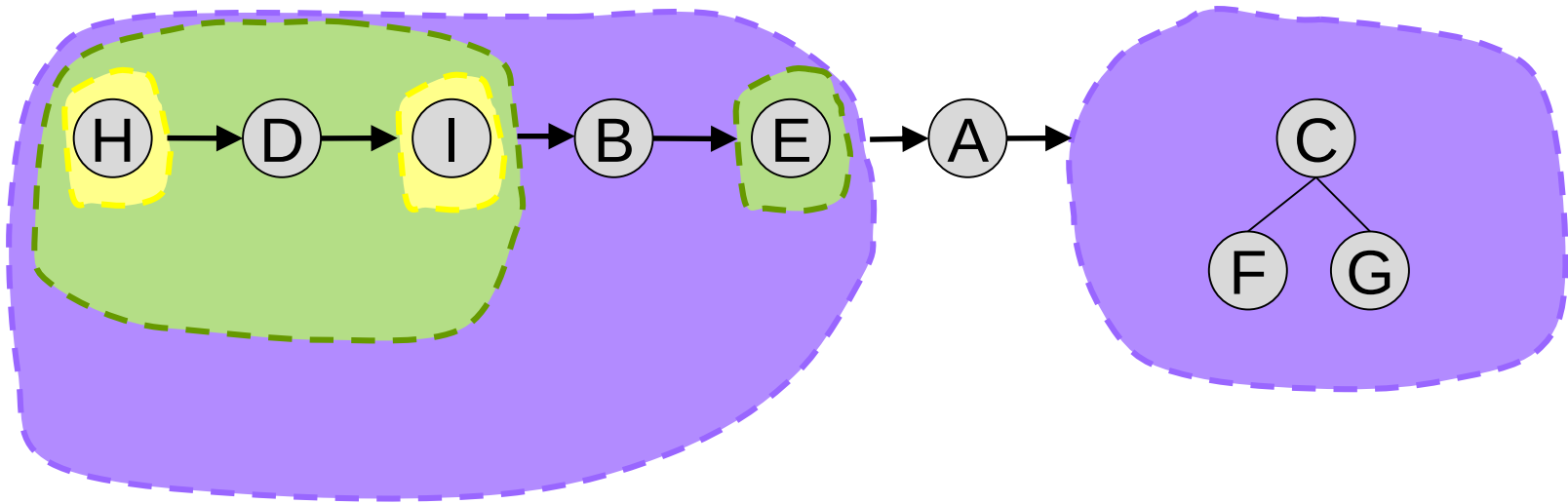
□ 예제: 중위 탐색 (Inorder) [2]



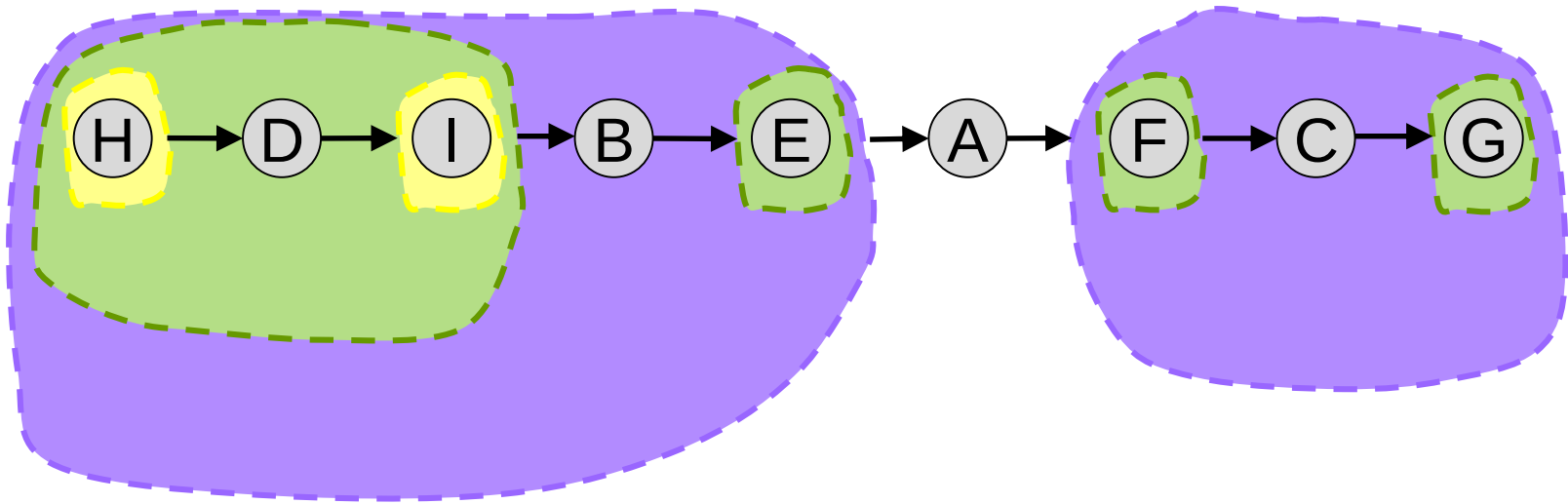
□ 예제: 중위 탐색 (Inorder) [3]



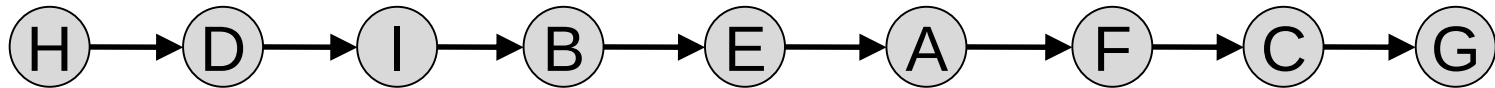
□ 예제: 중위 탐색 (Inorder) [4]



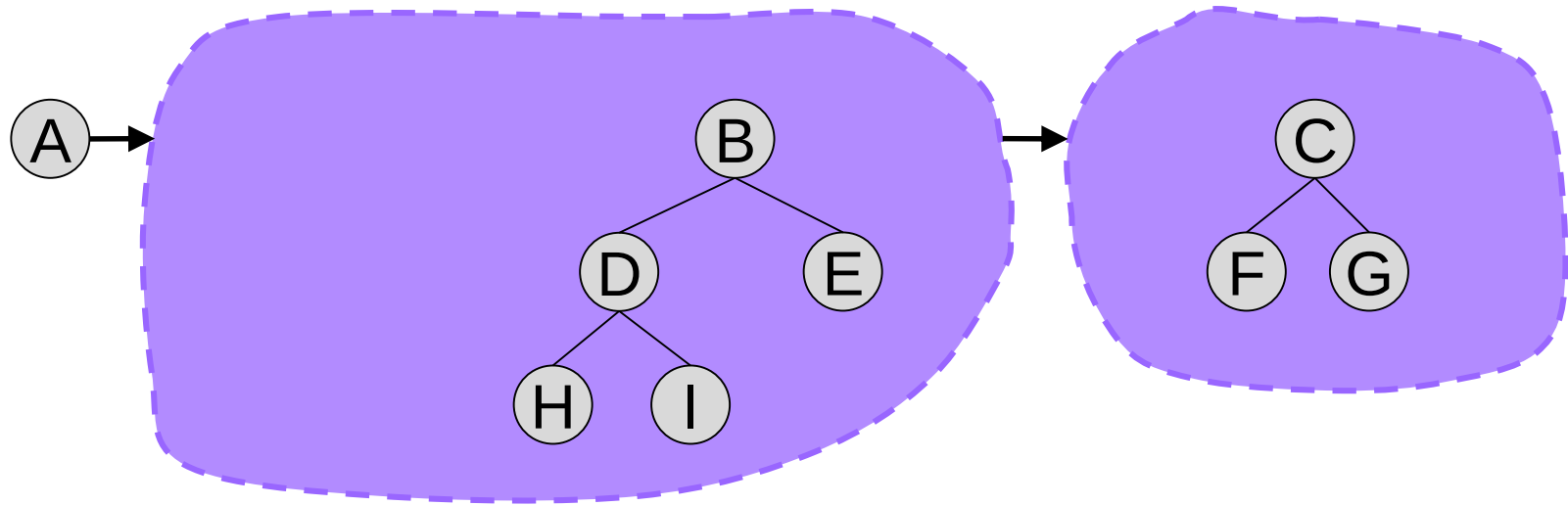
□ 예제: 중위 탐색 (Inorder) [5]



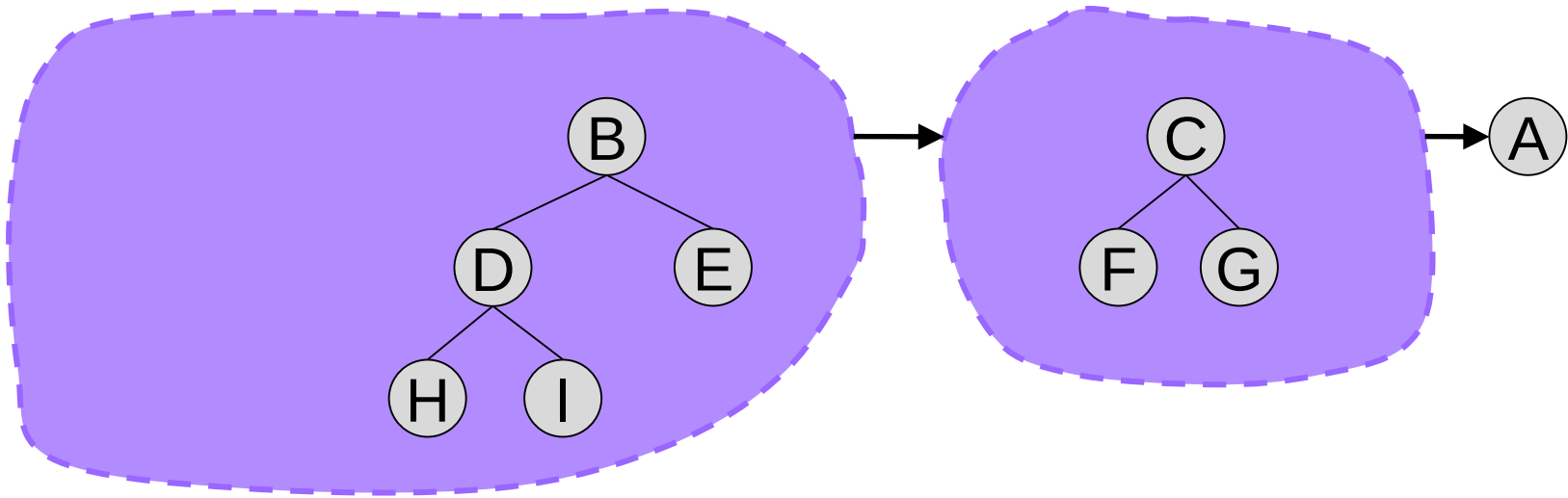
□ 예제: 중위 탐색 (Inorder) [6]



□ 예제: 전위 탐색 (Preorder)



□ 예제: 후위 탐색 (Postorder)



□ 탐색 알고리즘

■ 중위 탐색 (Inorder traversal)

```
private void inOrderRecursively (BinaryNode aRoot)
{
    if ( aRoot != NULL ) {
        this.inOrderRecursively (aRoot.leftChild() );
        this.visit (aRoot.element() );
        this.inOrderRecursively (aRoot.rightChild() );
    }
}
```

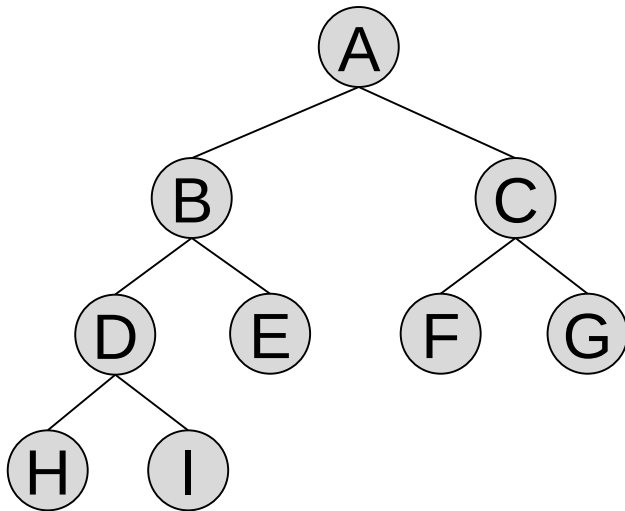
- 탐색은 재귀적으로(recursively) 실행된다.
- 그러므로, 코드에 보이지는 않지만 스택이 사용되고 있다.

□ 공개 함수 inOrder() 는?

```
public void inOrder ()  
{  
    this.inOrderRecursively (this._root) ;  
}
```

□ 레벨 순으로 탐색

```
Initially, add root to queue ;
While (queue is not empty) {
    X = remove node from queue ;
    visit node X ;
    add all children of node X to queue ;
}
```



Queue	Visit
→ A →	A
→ C → B →	B
→ E → D → C →	C
→ G → F → E → D →	D
→ I → H → G → F → E →	E
→ I → H → G → F →	F
→ I → H → G →	G
→ I → H →	H
→ I →	I
→ →	(End of Traversal)

□ 멤버 함수 levelOrder ()

```
public void levelOrder()
{
    Queue<BinaryNode> nodeQueue= new Queue() ;
    BinaryNode currentNode ;
    nodeQueue.add(this._root) ;
    while (! nodeQueue.isEmpty()) {
        currentNode = nodeQueue.remove() ;
        this.visit(currentNode.element());
        if ( currentNode.hasLeftChild() ) {
            nodeQueue.add(currentNode.leftChild() ) ;
        }
        if ( currentNode.hasRightChild() ) {
            nodeQueue.add(currentNode.rightChild() ) ;
        }
    }
}
```

Class "BinaryTree<T>"

□ BinaryTree<T>: 공개함수

■ BinaryTree 객체 사용법

- // 공개함수
- public BinaryTree () ;
- public BinaryTree (boolean shared,
 T givenRootElement,
 BinaryTree<T> givenLeftTree,
 BinaryTree<T> givenRightTree) ;
- public boolean isEmpty() ;
- public int height() ;
- public int size() ;
- public T rootElement() ;
- public BinaryTree<T> leftSubtree() ;
- public BinaryTree<T> rightSubtree() ;
- public void inorder() ;
- public void preOrder() ;
- public void postOrder() ;
- public void levelOrder() ;

□ BinaryTree<T>: 인스턴스 변수

```
public class BinaryTree<T>
{
    // 비공개 멤버 (인스턴스) 변수
    private BinaryNode<T> _root ;
```


□ BinaryTree<T>: 생성자

```
public class BinaryTree<T>
{
    // 생성자: empty tree를 생성
    public BinaryTree ( )
    {
        this._root = null ;
    }

    // 생성자: root 원소가 주어져서, 생성되는 트리가 empty 가 아닌 경우
    public BinaryTree ( boolean      shared ;
                       T              givenRootElement,
                       BinaryTree<T>  givenLeftTree,
                       BinaryTree<T>  givenRightTree)
    {
        if (shared) {
            this.setTreeByShare (givenRootElement, givenLeftTree, givenRightTree) ;
        }
        else {
            this.setTreeByCopy (givenRootElement, givenLeftTree, givenRightTree) ;
        }
    }
}
```

□ BinaryTree<T>: 상태 알아보기

```

public class BinaryTree<T>
{
    // 비공개 멤버 변수
    .....

    // 트리가 비어있는지 확인
    public boolean isEmpty()
    {
        return (this._root == null) ;
    }

    // 트리의 높이를 돌려준다
    public int height()
    {
        return heightOfSubtree (this._root) ;
    }

    private int heightOfSubtree (BinaryNode<T> aSubtreeRoot)
    {
        int height = 0 ;
        if ( aNode != null ) {
            int leftSubtreeHeight = this.heightOfSubtree(aSubtreeRoot.left()) ;
            int rightSubtreeHeight = this.heightOfSubtree(aSubtreeRoot.right()) ;
            int maxSubtreeHeight =
                (leftSubtreeHeight >= rightSubtreeHeight) ? leftSubtreeHeight : rightSubtreeHeight ;
            height = 1 + maxSubtreeHeight ;
        }
        return height ;
    }
}

```

□ BinaryTree<T>: 상태 알아보기

```
public class BinaryTree<T>
{
    // 비공개 멤버 변수
    .....

    // 트리의 노드 개수를 돌려준다
    public int size()
    {
        return sizeOfSubtree (this._root);
    }

    private int sizeOfSubtree (BinaryNode<T> aSubtreeRoot)
    {
        int size = 0 ;
        if ( aNode != null ) {
            int leftSubtreeSize = this.sizeOfTree (aSubtreeRoot.left()) ;
            int rightSubtreeSize = this.sizeOfTree (aSubtreeRoot.right()) ;
            size = 1 + (leftSubtreeSize + rightSubtreeSize) ;
        }
        return size;
    }
}
```

□ BinaryTree<T>: setTreeByShare()

```
public class BinaryTree<T>
{
    .....

    public void    setTreeByShare ( T          aRootElement,
                                   BinaryTree<T> aBinaryTreeForLeft,
                                   BinaryTree<T> aBinaryTreeForRight )
    {
        this._root = new BinaryNode<T>
            (aRootElement, aBinaryTreeForLeft._root, aBinaryTreeForRight._root) ;
    }
}
```

□ BinaryTree<T>: setTreeByCopy()

```
public class BinaryTree<T>
{
```

```
.....

public void    setTreeByCopy ( T          aRootElement,
                               BinaryTree<T> aBinaryTreeForLeft,
                               BinaryTree<T> aBinaryTreeForRight )
{
    T copiedRootElement = aRootElement.copy();
    BinaryNode<T> copiedRootOfLeftTree =
        copyBinaryTreeNodes (aBinaryTreeForLeft._root) ;
    BinaryNode<T> copiedRootOfRightTree =
        copyBinaryTreeNodes (aBinaryTreeForRight._root) ;

    this._root = new BinaryNode<T>
        (copiedRootElement, copiedRootOfLeftTree, copiedRootOfRightTree) ;
}
```

□ BinaryTree : 이진트리의 복사

```
public class BinaryTree<T>
{
    // 비공개 멤버 변수
    .....
    @Override
    private BinaryTree<T> copy ()
    {
        BinaryTree<T> copiedTree = new BinaryTree();
        if ( ! this.isEmpty() ) {
            BinaryNode<T> copiedLeftChild =
                copyBinaryTreeNodes(this._root.leftChild()) ;
            BinaryNode<T> copiedRightChild =
                copyBinaryTreeNodes(this._root.rightChild()) ;
            copiedTree._root = new BinaryNode<T>
                (this._root.element().copy(), copiedLeftChild, copiedRightChild) ;
        }
        return copiedTree;
    }
}
```

□ BinaryTree: 부트리 노드들의 재귀적 복사

```
public class BinaryTree<T>
{
    // 비공개 멤버 변수
    .....
    private BinaryNode<T> copyBinaryTreeNodes (BinaryNode<T> aNode)
    {
        if ( aNode == null ) {
            return null;
        }
        else {
            T copiedElement = aNode.element.copy();
            BinaryNode<T> copiedLeftChild =
                copyBinaryTreeNodes(aNode.left());
            BinaryNode<T> copiedRightChild =
                copyBinaryTreeNodes(aNode.right());
            return new BinaryNode<T>
                (copiedElement, copiedLeftChild, copiedRightChild);
        }
    }
}
```

"Call Back"

□ BinaryTree의 visit()는 어떻게 구현하면 좋을까?

- 예: 중위 탐색 (Inorder traversal)

```
public class BinaryTree<T> {  
    .....  
  
    private void visit (T anElement)  
    {  
        ..... // 여기를 어떻게?  
    }  
  
    private void inOrderRecursively (BinaryNode<T> aRoot)  
    {  
        if ( aRoot != NULL ) {  
            this.inOrderRecursively (aRoot.leftChild());  
            this.visit (aRoot.element());  
            this.inOrderRecursively (aRoot.rightChild());  
        }  
    }  
    .....  
}
```

□ 사용자가 visit() 기능을 정의하는 방법은?

- “BinaryTree” 객체를 사용하는 사용자는 자신의 용도에 맞게 visit() 를 정의하고 싶다.
- 무엇이 문제인가?
 - inOrder()는 Class “BinaryTree”의 public method
 - visit()는 “BinaryTree” 내부에 정의되며, inOrder() 안에서 사용한다.
 - ◆ 엄밀히는, inOrder() 가 call 하는 inOrderRecursively() 안에서.
- 그러므로, inOrder()를 call 하는 쪽 즉 “BinaryTree” 객체의 사용자는, Class “BinaryTree” 안에 private 하게 정의되고 구현되는 (그래서 캡슐화 되는) visit() 의 실행 내용을 임의로 변경할 수 없다!

□ "visit()" 의 실질적인 내용은 사용자가 정의!

- "BinaryTree"의 visit()에서 실제로 해야 할 일을 사용자의 클래스에 정의하고 구현한다.
 - 사용자가 ApplicationController 객체라면, Class "AppController" 안에.
- visit() 의 정의 자체는 당연히 가능하다.
- 다만, 이 내용을 Binary Tree 객체로 하여금 어떻게 알게 할 수 있을까?

- "visit()" 의 실질적인 내용을 BinaryTree 객체에게 알린다!
- 사용자의 클래스에 정의된 이 함수를 "BinaryTree" 객체에게 알려준다.
 - 사용자가 inOrder() 를 시행하기 전에, 자신이 사용할 "BinaryTree" 객체에게 스스로 자신이 누구인지를, 미리 알려준다.
- 사용자는 자신이 누구인지를 아는 "BinaryTree" 객체에게 inOrder() 를 실행하게 한다.
- Call Back
 - "BinaryTree" 객체는, visit() 를 실행하게 되면, 자신에게 일을 시킨 사용자 객체를 알고 있으므로, 사용자 객체가 정의한 일을 "call Back" 한다.

“Call Back” 의 구현

□ Call Back 할 함수는 Interface로 정의하자

■ Call Back 함수

- 사용자 class에 정의되어 있지만, 피사용자 객체가 call 하게 되는 함수

```
public interface VisitEventForTreeTraversal
{
    public void visitByCallback (Object anObject);
}
```

■ 사용자와 "BinaryTree"가 서로 알고 있어야 할 함수를 Interface로 정의한다.

- 정의와 구현은 사용자 클래스에서
- 사용 (call back) 은 피사용자 클래스 (BinaryTree) 에서

■ 여기서는 "visitByCallback()" 이 call back 함수이다.

□ 사용자 쪽의 선언

- Call Back을 위한 Interface는 당연히 사용자 클래스에서 구현

```
public class AppController implements VisitEventForTreeTraversal
{
    .....
    BinaryTree<T> _binaryTree = new BinaryTree<T>() ;
    .....
    @Override
    public void visitByCallback (T anElement) {
        // TODO Auto-generated method stub
        ..... // visit 에서 실제로 할 일
    }
    .....
    // "BinaryTree" 객체에게 사용자가 누구인지를 알려준다.
    this._binaryTree.setCallerForVisitEvent (this);
    this._binaryTree.inorder() ; // "BinaryTree" 객체에게 탐색을 하게 한다.
    .....
}
```

□ 피사용자인 “BinaryTree” 쪽은?

```
public class BinaryTree<T>
{
    private VisitEventForTreeTraversal _callerForVisitEvent ;
        // VisitEventForTreeTraversal 를 구현한 임의의 Class의 객체를 의미한다
    .....
    public void setCallerForVisitEvent (VisitEventForTreeTraversal newCaller)
    { // Setter for _callerForVisitEvent
        this._callerForVisitEvent = newCaller;
    }
    .....
    private void visit (T anElement)
    {
        this._callerForVisitEvent.visitByCallBack (anElement) ;
    }
    .....
}
```


□ 이진트리의 Traversal 을 구현할 때 생각해야 할 점

- 이진트리의 탐색은 구현에 의존적이다.
 - 그러므로, 탐색 자체는 이진 트리 내부에 구현되어야 할 일이다.
- 그러나,

특정 노드를 방문했을 때 해야 할 일은, 이진트리의 사용자가 정의할 수 있어야 한다. 즉, 이진 트리를 사용하는 사람의 용도에 맞게 표현되어야 한다. 그렇다는 것은, 방문했을 때 해야 할 일을 이진 트리를 구현하는 class 내부에 정의할 수는 없다.

 - 이진 트리 class 내부에 구현하는 방법으로 해결하는 것은, Model-View-Controller 의 개념을 깨뜨리게 될 위험성이 높다.
 - ◆ 방문 시에 해야 할 일이 출력할 일이라면.....
 - 또한 고정시켜 버린 방문(visit) 행위는, "DictionaryByBinarySearchTree" 객체를 사용하는 또 다른 사용자가 자신의 목적에 맞는 용도로 사용할 수 없게 만든다. 아니면 각 사용자는 자신 만의 방문 행위를 포함하는 자신 만의 목적에 맞는 "DictionaryByBinarySearchTree" class 를 정의하여 사용해야 할 것이다.

□ 문제의 요약 [1]

■ 주어진 상황:

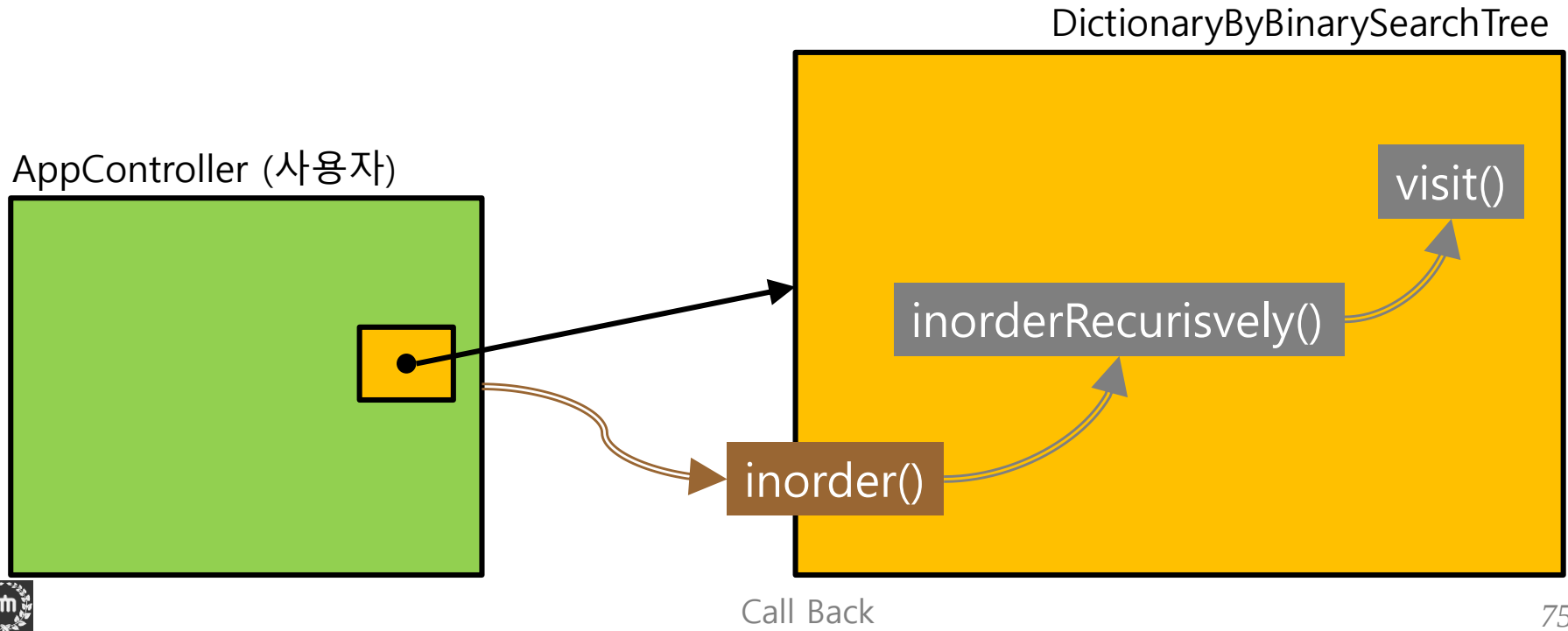
- 우리가 사용할 이진트리를 구현하는 class는 "DictionaryByBinarySearchTree" 이다.
- 이진트리 탐색은 inorder traverse (중위 탐색) 을 실행한다:

```
public class DictionaryByBinarySearchTree<...> extends Dictionary<...>
{
    .....

    private void inorderRecursively (BinaryNode<...> aRootOfSubTree) {
        if ( aRootOfSubTree != null ) {
            this.inorderRecursively (aRootOfSubTree.left()) ;
            this.visit() ;
            this.inorderRecursively (aRootOfSubTree.right()) ;
        }
    }
    public void inOrder() {
        this.inorderRecursively (this.root());
    }
} // End of class "DictionaryByBinarySearchTree"
```

□ 문제의 요약 [2]

- "visit()"는 class "DictionaryByBinarySearchTree"의 private method.
- ApplicationController로서는 "visit()"의 내용을 자신의 목적에 맞게 구현할 수 있을까?



□ 문제의 요약 [3]

■ 문제점:

- "inorderRecursively()" 내부에서 call 하는 함수 "visit()" 는, **형식적인 관점에**서 자신의 class "DictionaryByBinarySearchTree" 안에 정의하는 것이 보통의 방법일 것이다.
- 그러나 **실제적인 관점에서는** 함수 "visit()"의 내용은, class "DictionaryByBinarySearchTree" 객체의 **사용자가 자신의 목적에 맞게** 정의할 수 밖에 없다.
- 또한 사용자마다 사용 목적이 다를 수 있으므로 "visit()" 의 내용이 달라질 것이다.
- 그렇다면, inorder traverse 의 사용자에게, 자신의 원하는 바를 class "DictionaryByBinarySearchTree" 내부의 함수 visit() 를 직접 구현할 수 있게 허락해야 할까? 허락하면 어떤 문제가 있을까? 허락하지 않는다면 해결책은?

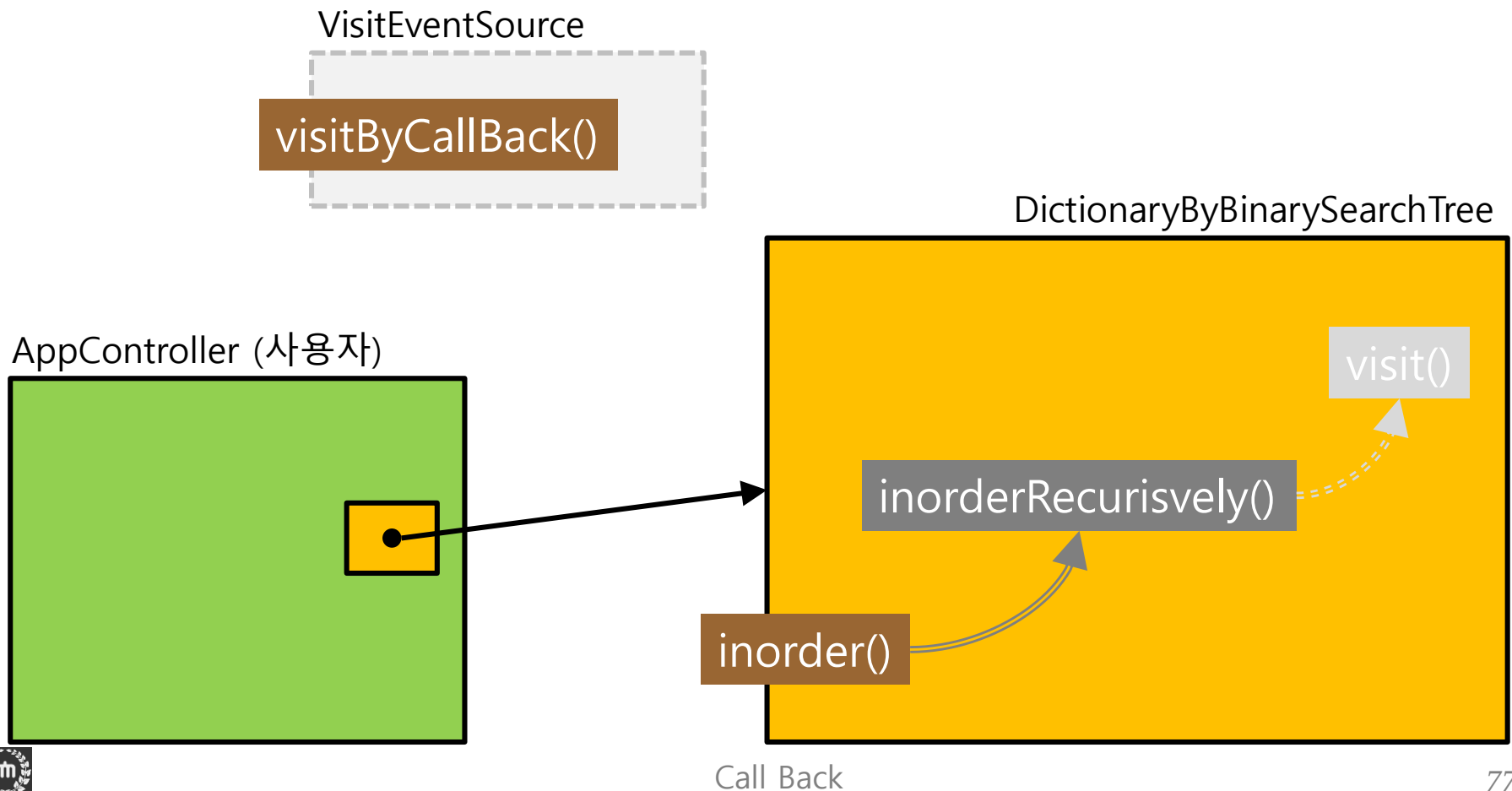
■ 해결책:

- "visit()"의 구현은 사용자가, 실행은 "DictionaryByBinarySearchTree" 객체가 !!
- 어떻게?



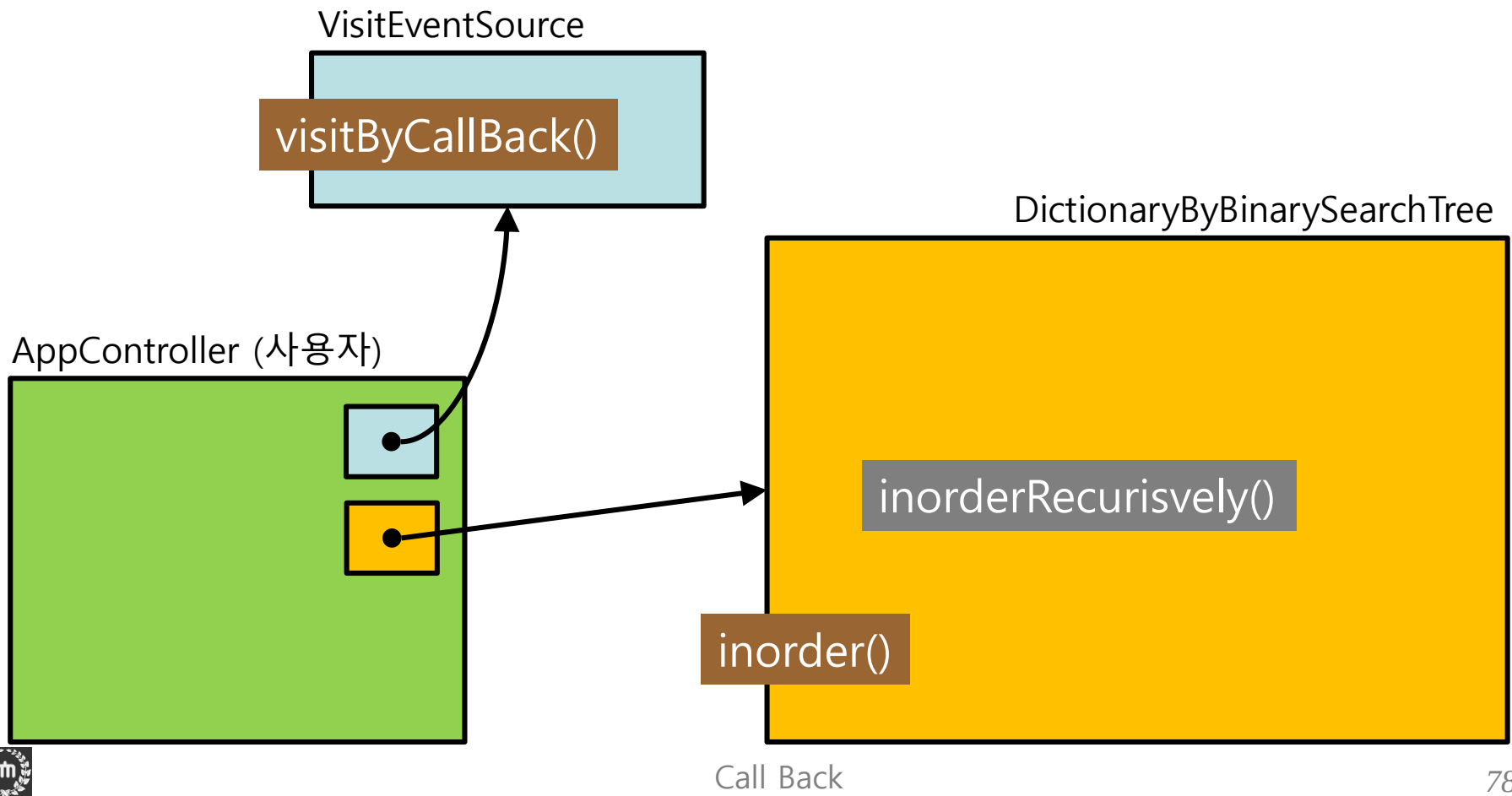
□ 해결책은? [1]

- "DictionaryByBinarySearchTree" 는 "VisitEventSource"를 **Java Interface** 로 정의하여 사용자에게 제공한다. 즉 함수 "**visitByCallBack()**" 의 사용법만 정의하게 된다. 결국 "visitByCallBack()" 이 "visit()"의 역할을 하게 된다.



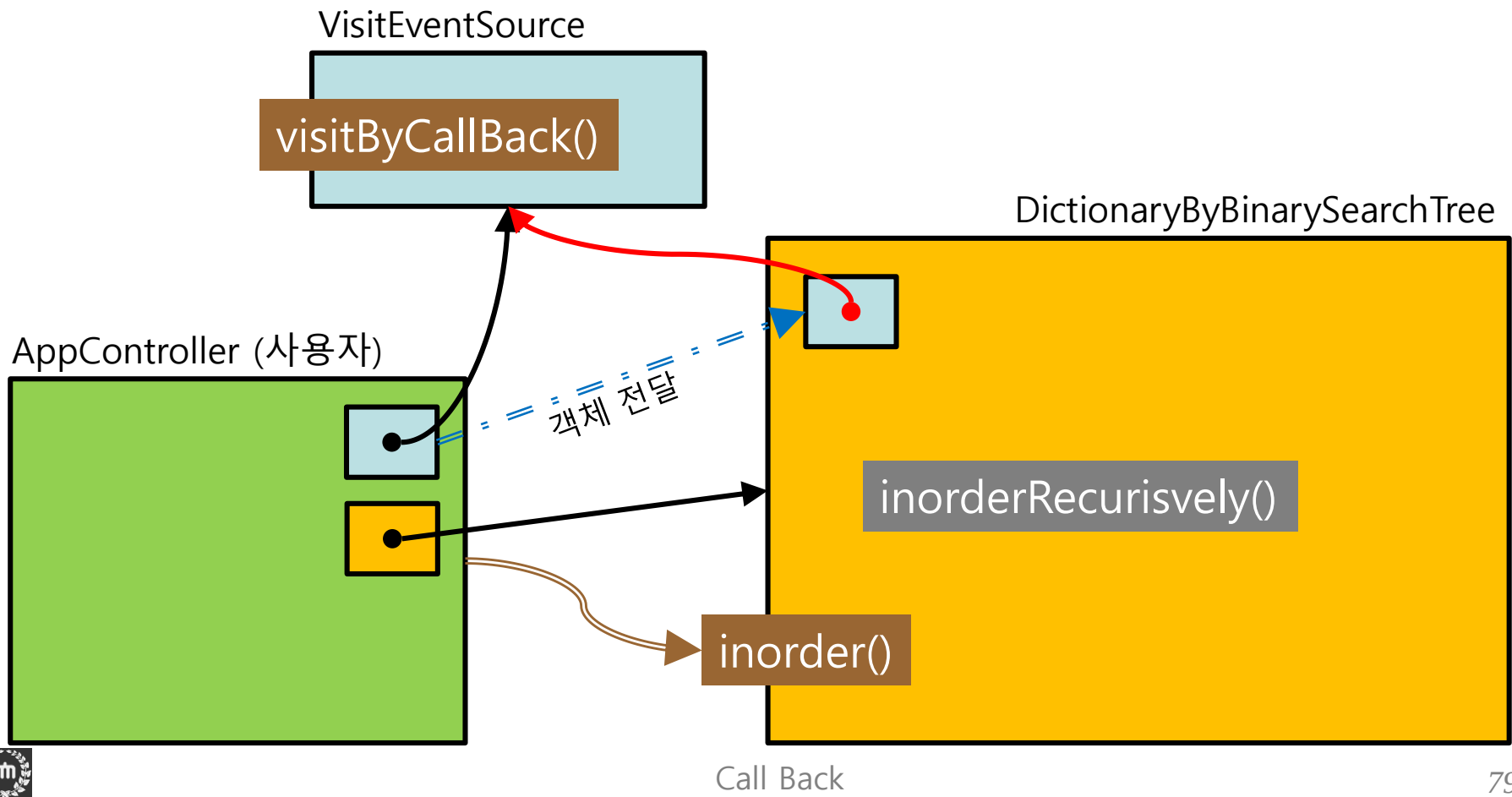
□ 해결책은? [2]

- 사용자는 인터페이스 "VisitEventSource" 를 구현해 놓는다.
- 사용하는 시점에 사용자는 "VisitEventSource" 객체를 생성한다.



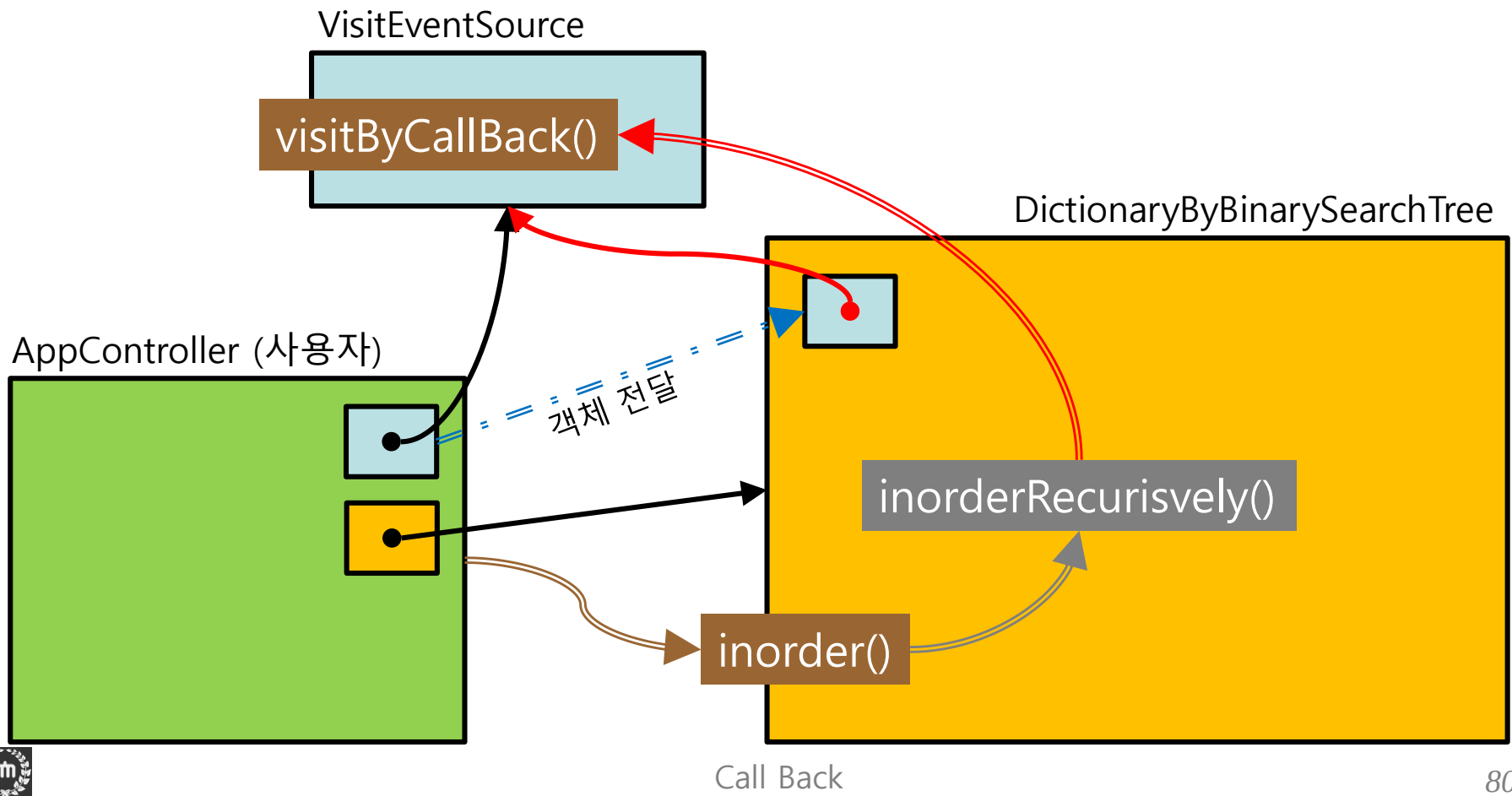
□ 해결책은? [3]

- 사용자는 "VisitEventSource" 객체를 "DictionaryByBinarySearchTree" 객체로 전달한다.
- 사용자는 "DictionaryByBinarySearchTree" 객체에게 "inorder()"를 실행하게 한다.



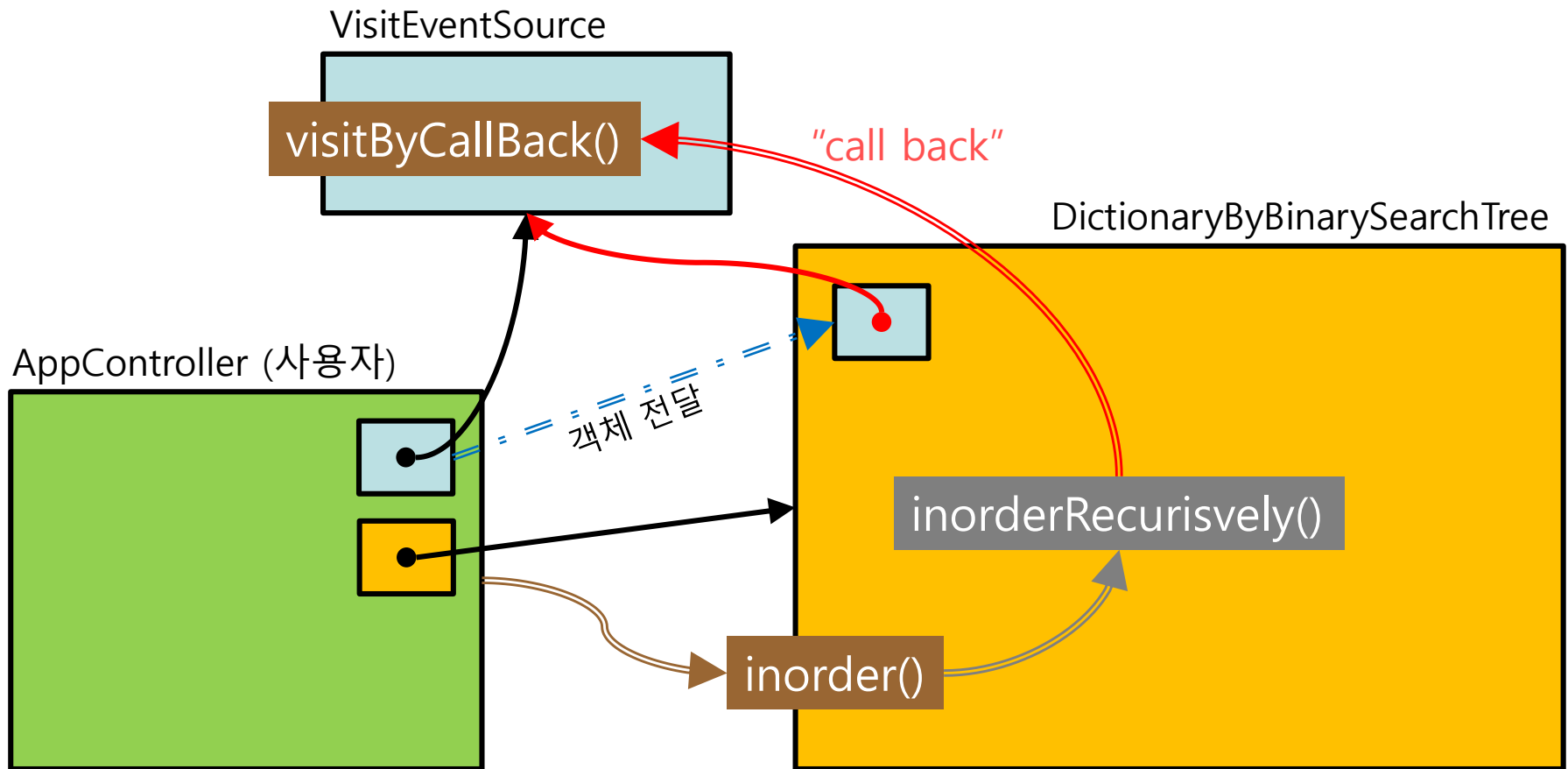
□ 해결책은? [4]

- "DictionaryByBinarySearchTree" 객체는 전달받은 "VisitEventSource" 객체에게 "visitByCallBack()"을 실행하게 한다.
- 이로써, <visit의 행위는 사용자가 정의하고, 실행은 "DictionaryByBinarySearchTree"가 한다>는 우리의 목적이 달성될 수 있다.



□ Call Back?

- AppController 객체는 VisitEventSource 객체를 생성하여 소유하고 있다. 그리고 사전 (DictionaryByBinarySearchTree) 객체에게 "inorder()" 라는 일을 시키려고 call 하고 있다. 사전 객체는 자신의 사용자인 AppController 객체가 소유한, 그리고 지금 자신에게 전달한 VisitEventSource 객체에게 일을 시킨다. 이 행위는 자신을 call 한 객체 쪽으로 되돌아call 을 하는, 즉 **call back** 하는 셈이 된다.



Interface "VisitEventSourceForTreeTraversal"



□ Interface VisitEventSource

- Class "DictionaryByBinarySearchTree"의 트리 탐색 (traverse)을 실행하는 함수에서 (예를 들어 inorder()), 노드를 방문(visit) 할 때 마다 실행할 Callback 함수 사용법 (함수의 이름과 매개변수)를 정의하기 위한 Interface.

```
public interface VisitEventSource<Key,Obj> {
    public void visitForInorder (DictionaryElement<Key,Obj> anElement, int aLevel) ;
    public void visitForReverseOfInorder
        (DictionaryElement<Key,Obj> anElement, int aLevel) ;
}
```

- Class "DictionaryByBinarySearchTree"와 사용자 class 는 Callback 을 위한 interface "VisitEventSource"를 공유한다.
- Callback 함수("visitForInorder()" 와 "visitForReverseOfInorder()")의 매개변수:
 - ◆ 사용자는 트리를 탐색하는 동안 노드를 방문할 때 마다, 방문 노드의 원소 (anElement)와 그 노드의 트리 레벨 (aLevel) 정보를 Callback 함수를 통해서 전달받게 된다.
 - ◆ 이 두 정보를 사용하여 사용자는 자신의 목적에 맞는 일을 구현할 수 있다.

Class "DictionaryByBinarySearchTree" 의 수정



□ DictionaryByBinarySearchTree의 수정 [1]

```

public class DictionaryByBinarySearchTree <Key extends Key, Obj> extends Dictionary<Key,Obj> {
    ....

    private VisitEventForDictionaryByBinarySearchTree _visitEvent ;
        // 전달받은 Visit event 객체를 저장할 곳

    public VisitEventForDictionaryByBinarySearchTree visitEvent () {
        return this._visitEvent ;
    }
    public void setVisitEvent (VisitEventForDictionaryByBinarySearchTree newVisitEvent) {
        this._visitEvent = newVisitEvent ;
    }

    .....

    private void inorderRecursively (
        BinaryNode<DictionaryElement<Key,Obj> > aRootOfSubtree,
        int aLevel )
    {
        if ( aRootOfSubtree != null ) {
            this.inorderRecursively (aRootOfSubtree.left(), aLevel+1) ;
            this.visitEvent().visitForInorder (aRootOfSubtree.element(), aLevel) ;
            this.inorderRecursively (aRootOfSubtree.right(), aLevel+1) ;
        }
    }

    public void inorder () {
        this.inorderRecursively (this.root(), 1) ;
    }
}

```

□ DictionaryByBinarySearchTree의 수정 [2]

```

public class DictionaryByBinarySearchTree <Key extends Key, Obj> extends Dictionary<Key,Obj> {
    ....

    private void inorderRecursively (...) {...}
    public void inorder () {...}

    private void reverseOfInorderRecursively (
        BinaryNode<DictionaryElement<Key,Obj> > aRootOfSubtree, int aLevel)
    {
        if ( aRootOfSubtree != null ) {
            this.reverseOfInorderRecursively (aRootOfSubtree.right(), aLevel+1) ;
            this.visitEvent().visitForInorder (aRootOfSubtree.element(), aLevel) ;
            this.reverseOfInorderRecursively (aRootOfSubtree.left(), aLevel+1) ;
        }
    }
    public void reverseOfInorder () {
        this.reverseOfInorderRecursively (this.root(), 1) ;
    }
}

```

Class "AppController" 의 수정



□ ApplicationController: Call Back 관련 수정[1]

```
public class ApplicationController implements VisitEventSourceForTreeTraversal<Integer, Integer>
{
```

```
// Constants
private static final int
    DEFAULT_DATA_SIZE = 10 ;
```

```
// Private instance variables
private AppView _appView ;
private DictionaryByBinarySearchTree<Integer,Integer> _dictionary ;
private int[] _list ;
```

```
...
```

```
public void run() {
    this._list = DataGenerator.randomList(DEFAULT_DATA_SIZE) ;

    this._appView.outputLine(...) ;
    this._dictionary = new DictionaryByBinarySearchTree<Integer, Integer>() ;
```

```
    this._dictionary.setVisitEventSource (this) ;
```

```
    this.addToBinarySearchTreeAndShowShape() ;
    this.showInorderOfBinarySearchTree() ;
    this.removeFromBinarySearchTreeANdShowShape() ;
}
```

별도의 event class 를 정의하여, event source 객체를 생성하는 방법을 사용할 수도 있다. 그러나 여기서는 그렇게 하지 않고, ApplicationController 자신을 event source 객체로 하는 방법을 사용한다.

Dictionary 객체에게 Visit event의 source 가 어느 객체인지를 알려 준다. 여기서는 ApplicationController 객체 자신이므로 **this** 를 전달한다.

□ ApplicationController: Call Back 관련 수정[2]

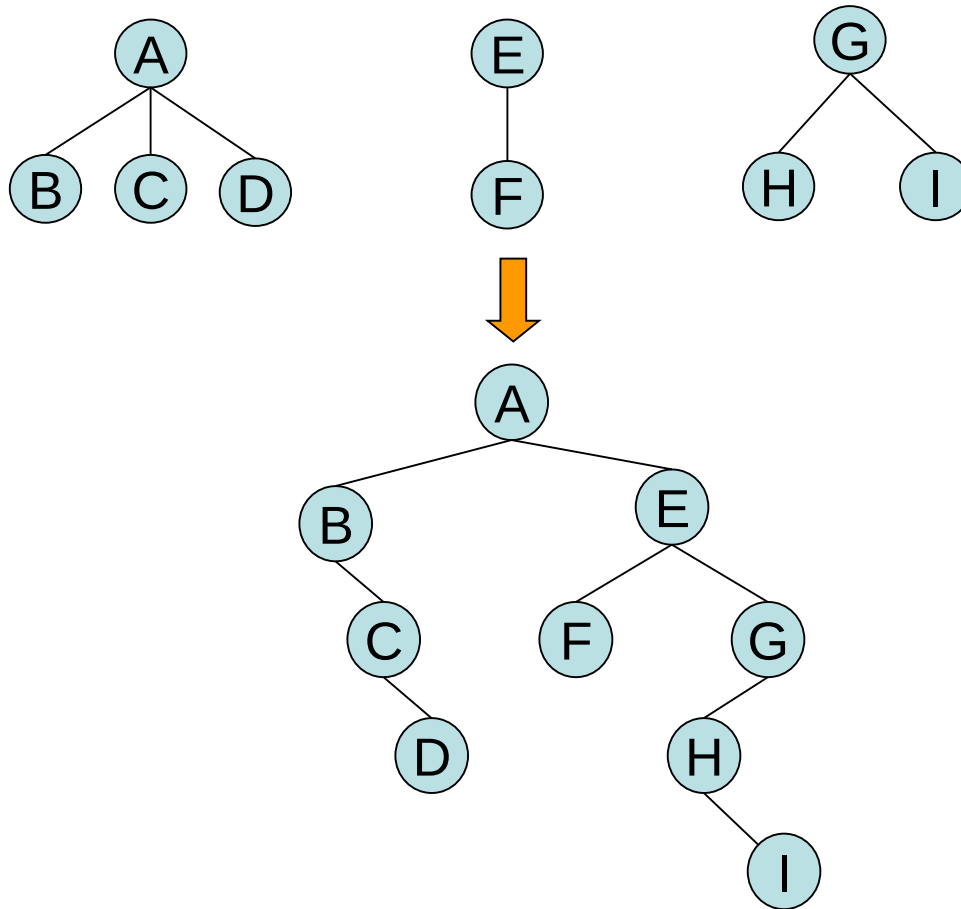
```
public class ApplicationController {  
  
    .....  
  
    public void run() {.....}  
  
    @Override  
    public void visitForInorder (DictionaryElement<Integer, Integer> anElement, int aLevel)  
    {  
        // TODO Auto-generated method stub  
        ..... // Inorder 탐색을 하는 동안 원소를 방문했을 때 해야 할 일  
    }  
  
    @Override  
    public void visitFoReverseOfInorder (DictionaryElement<Integer, Integer> anElement, int aLevel)  
    {  
        // TODO Auto-generated method stub  
        ..... // Inorder 의 역순으로 탐색하는 동안 원소를 방문했을 때 해야 할 일  
    }  
}
```



Traversals for Forests

□ 숲 (Forests)

- 숲 (forest): 0 개 이상의 서로 원소가 겹치지 않는 트리들의 집합.
- 숲을 이진트리로 변환:



□ 숲의 전위 탐색

■ $F = \{ T_1, T_2, \dots, T_n \}$

■ Tree Preorder (F)

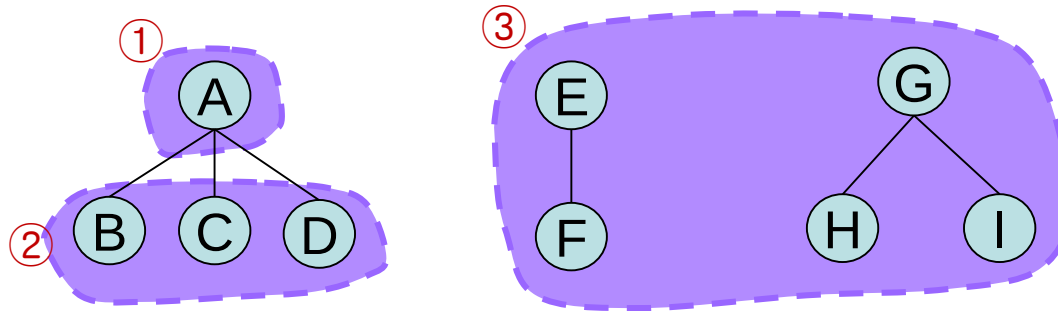
If (F 가 공집합이 아니라면) {

 T1의 루트를 방문 ;

 T1의 의 부트리들을 Tree Preorder 로 탐색 ;

 F의 나머지 트리들(T2 , ... , Tn) 을 Tree Preorder 로 탐색 ;

}



■ 예:

● 숲:

A - B - C - D - E - F - G - H - I

● 이진 트리:

A - B - C - D - E - F - G - H - I

□ 숲의 중위 탐색

■ $F = \{ T_1, T_2, \dots, T_n \}$

■ Tree Inorder (F)

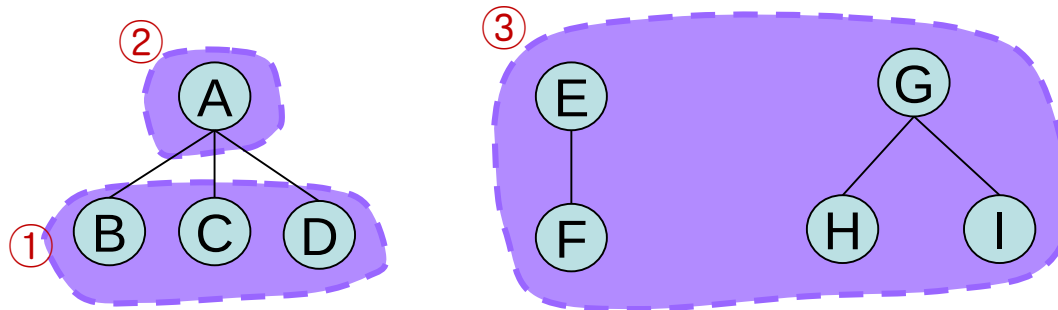
If (F 가 공집합이 아니라면) {

 T1의 의 부트리들을 Tree Inorder 로 탐색 ;

 T1의 루트를 방문 ;

 F의 나머지 트리들(T2 , ... , Tn) 을 Tree Inorder 로 탐색 ;

}



■ 예:

● Forest : B – C – D – A – F – E – H – I – G

● Binary Tree : B – C – D – A – F – E – H – I – G

Postorder Traverse of Forests

■ $F = \{ T_1, T_2, \dots, T_n \}$

■ Postorder (F)

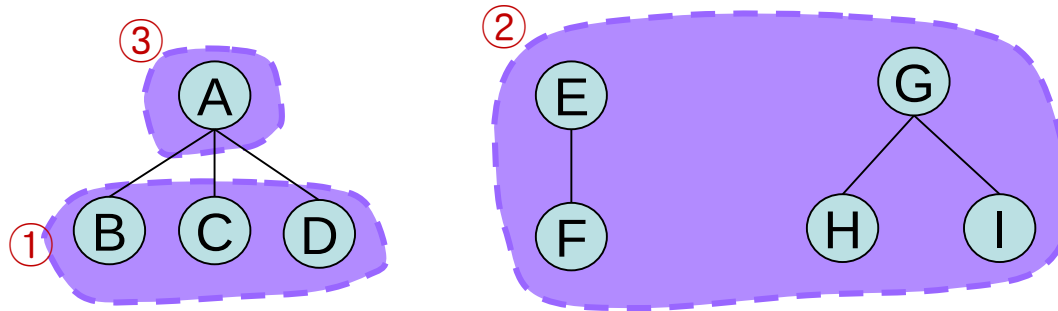
If (F 가 공집합이 아니라면) {

 T1의 의 부트리들을 Tree Postorder 로 탐색 ;

 F의 나머지 트리들(T2 , ... , Tn) 을 Tree Postorder 로 탐색 ;

 T1의 루트를 방문 ;

}



■ 예:

● Forest : D – C – B – F – I – H – G – E – A

● Binary Tree : D – C – B – F – I – H – G – E – A

“Tree” [끝]

